# VulnScan – Intelligent Web Application Vulnerability Assessment Platform with Risk Assessment and Remediation Knowledge Base

By

OOI JIAN XIN




FACULTY OF COMPUTING AND

INFORMATION TECHNOLOGY

TUNKU ABDUL RAHMAN UNIVERSITY OF MANAGEMENT AND TECHNOLOGY

KUALA LUMPUR

ACADEMIC YEAR

2025/26

# VulnScan – Intelligent Web Application Vulnerability Assessment Platform with Risk Assessment and Remediation Knowledge Base

By

OOI JIAN XIN

Supervisor: Dr Muhammad Naim Bin Abdullah

A project report submitted to the

Faculty of Computing and Information Technology

in partial fulfillment of the requirement for the

Bachelor of Information Technology (Honours) in Information Security

Faculty of Computing and Information Technology

Tunku Abdul Rahman University of Management and Technology

Kuala Lumpur

**Copyright by Tunku Abdul Rahman University of Management and Technology.**

All rights reserved. No part of this project documentation may be reproduced, stored in retrieval system, or transmitted in any form or by any means without prior permission of Tunku Abdul Rahman University of Management and Technology.

# Declaration

The project submitted herewith is a result of my own efforts in totality and in every aspect of the project works. All information that has been obtained from other sources had been fully acknowledged. I understand that any plagiarism, cheating or collusion or any sorts constitutes a breach of TAR University rules and regulations and would be subjected to disciplinary actions.

_____

OOI JIAN XIN

ID: 25WMR09764

# Abstract

Modern web applications have become essential platforms for delivering online services across various industries. However, their increasing complexity has also introduced numerous security vulnerabilities that may lead to unauthorized access, data breaches, and service disruption. Although many vulnerability assessment tools are available, existing solutions are often designed for experienced cybersecurity professionals and primarily focus on vulnerability detection while providing limited educational guidance and remediation support. This project proposes VulnScan, an Intelligent Web Application Vulnerability Assessment Platform with Risk Assessment and Remediation Knowledge Base, to provide an accessible and educational solution for identifying and understanding common web application security vulnerabilities. The proposed system enables developers to perform automated vulnerability assessments by scanning target web applications for selected vulnerabilities, including SQL Injection (SQL Injection), Cross-Site Scripting (XSS), and HTTP Security Header misconfigurations. In addition to vulnerability detection, the system incorporates configurable scan profiles, risk assessment, role-based access control, security audit logging, and an integrated knowledge base containing vulnerability descriptions, security impacts, secure coding practices, and remediation recommendations. The system adopts the Agile Software Development methodology, allowing iterative planning, design, implementation, testing, and continuous improvement throughout the project lifecycle. The system architecture follows a layered design consisting of the Presentation Layer, REST API Layer, Business Logic Layer, Scanner Engine, and Data Layer to improve modularity, scalability, and maintainability. Database normalization and modular vulnerability management further support future system expansion. Functional and non-functional testing are conducted to verify that the implemented system satisfies the specified requirements and performs reliably under normal operating conditions. Overall, VulnScan demonstrates that integrating automated vulnerability assessment with educational resources, centralized management, and risk evaluation can improve both web application security and users' understanding of secure software development. The proposed platform provides a practical and scalable foundation for future enhancement through additional vulnerability detection modules and advanced security analysis capabilities.

# Acknowledgement

First and foremost, I would like to express my sincere gratitude to my project supervisor, Dr Muhammad Naim Bin Abdullah, for his invaluable guidance, continuous encouragement, and constructive feedback throughout the development of this Final Year Project. His expertise and advice have greatly contributed to the successful completion of this project.

I would also like to extend my appreciation to the lecturers of the Faculty of Computing and Information Technology, Tunku Abdul Rahman University of Management and Technology, for providing the knowledge, skills, and learning opportunities that have prepared me for undertaking this project.

My heartfelt thanks go to my family and friends for their continuous support, patience, and encouragement throughout my academic journey. Their understanding and motivation have enabled me to overcome the challenges encountered during the development of this project.

Finally, I would like to thank everyone who has directly or indirectly contributed to the completion of this project. Their support and assistance are sincerely appreciated.

# Table of Contents


**1.0 Introduction** **2**

1.1 Background and Context 2

1.1.1 Cybersecurity 2

1.1.2 Web Applications 2

1.1.3 Web Vulnerabilities 3

1.1.4 Vulnerability Assessment 3

1.1.5 Existing Problems 3

1.1.6 Our System 4

1.2 Problem Statement 4

1.2.1 Complexity of Existing Tools 4

1.2.2 Lack of Integrated Risk Assessment and Remediation Guidance 5

1.2.3 Limited Flexibility and Centralized Management 5

1.2.4 Proposed Solution 5

1.3 Objectives of the Project 6

1.4 Scope and Limitations 7

1.4.1 Functional Scope 7

1.4.2 Project Limitations 8

1.5 Significance of the Project 9

1.6 Organisation of Report 10

1.7 Chapter Summary and Evaluation 11

**2.0 Literature Review** **13**

2.1 Related Theories and Concepts 13

2.1.1 Web Application Security 13

2.1.2 Vulnerability Assessment 14

2.1.3 OWASP Top 10 15

2.1.4 Risk Assessment 16

2.1.5 Secure Coding Practices 18

2.1.6 Role-Based Access Control (RBAC) 19

2.2 Study of Similar Existing Systems 20

2.2.1 Review of Similar Existing System 20

2.2.2 Comparison of These Systems 23

2.2.3 Analysis of User Feedback, Adoption Rate, and Success Stories 25

2.3 Research Gap 27

2.4 Summary of Key Findings 29

2.5 Chapter Summary and Evaluation 30

**3.0 Methodology and Requirements Analysis** **32**

3.1 Project Methodology 32

3.1.1 Planning Phase 34
3.1.2 Analysis Phase 35
3.1.3 Design Phases 35
3.1.4 Implementation Phase 36
3.1.5 Testing and Integration 36
3.1.6 Maintenance 37
3.2 Requirement Gathering 38
3.2.1 Existing System Analysis 38
3.2.2 Functional Requirements 39
3.2.3 Non-Functional Requirements 40
3.3 Requirement Analysis 42
3.3.1 Analysis and Prioritization of Requirements 42
3.3.2 Identification of Dependencies and Conflicts among Requirements 43
3.4 Summary 44
**4.0 System Design 46**
4.1 Architecture Design 46
4.2 Database Design 48
4.3 User Interface Design 51
4.4 Detailed Design 56
4.4.1 User Authentication Process 56
4.4.2 Vulnerability Scanning Process 58
4.4.3 Security Audit Logging Process 60
4.5 Chapter Summary and Evaluation 62
**5.0 Implementation and Testing 64**
5.1 Sub-section 1 Heading 64
Sub-subsection Heading 64
5.2 Sub-section 2 Heading 65
Sub-section 1 Heading 65
Sub-subsection Heading 65
5.3 Sub-section 1 Heading 65
Sub-subsection Heading 65
5.4 Chapter Summary and Evaluation 65
**6.0 System Deployment 67**
System Backup and Risk Management 67
On-site Setup 67
Training Procedure 67
Follow-up 67
Chapter Summary and Evaluation 67
**7.0 Discussions and Conclusion 69**

Summary 69
Achievements 69
Contributions 69
Limitations and Future Improvements 69
Issues and Solutions 70
**References** **71**

Chapter 1

# Introduction

# 1.0 Introduction

This chapter introduces the proposed project, VulnScan: An Intelligent Web Application Vulnerability Assessment Platform with Risk Assessment and Remediation Knowledge Base. It presents the background and context of web application security, discusses the problems that motivate the development of the proposed system, and outlines the project's objectives, scope, limitations, and significance. In addition, this chapter provides an overview of the report structure to guide readers through the subsequent chapters. The purpose of this chapter is to establish a clear understanding of the project's motivation, research direction, and expected contributions in addressing current web application security challenges.

## 1.1 Background and Context

### 1.1.1 Cybersecurity

The rapid growth of digital technologies has transformed the way organizations, businesses, educational institutions, and government agencies deliver services through web applications. Online platforms are now widely used for banking, healthcare, education, e-commerce, and cloud-based services, enabling users to access information and perform transactions conveniently from anywhere with an Internet connection. As web applications continue to store and process sensitive information such as personal records, financial data, authentication credentials, and confidential business information, ensuring the security of these applications has become one of the most critical concerns in modern cybersecurity (OWASP Foundation, 2021; NIST, 2020).

### 1.1.2 Web Applications

Despite significant advancements in software development practices, web applications remain one of the most frequently targeted attack surfaces by cybercriminals. Security vulnerabilities introduced during software development, insecure configurations, outdated software components, and improper input validation expose web applications to numerous cyberattacks. Successful exploitation of these vulnerabilities may result in unauthorized access, data breaches, privilege escalation, website defacement, service disruption, or complete system compromise (OWASP Foundation, 2021). Consequently, organizations increasingly recognize that identifying and mitigating vulnerabilities before deployment is essential to reducing cybersecurity risks.

### 1.1.3 Web Vulnerabilities

To address these security challenges, vulnerability assessment has become a fundamental practice in secure software development and cybersecurity risk management. Vulnerability assessment involves systematically examining web applications to identify security weaknesses that could potentially be exploited by attackers. Automated vulnerability scanners have become widely adopted because they significantly reduce the time and effort required to discover common security flaws compared to manual penetration testing (Scambray et al., 2021). . These tools assist developers and security professionals in detecting vulnerabilities during the software development lifecycle, enabling security issues to be remediated before they become exploitable.

### 1.1.4 Vulnerability Assessment

The Open Worldwide Application Security Project (OWASP) provides one of the most widely recognized standards for web application security through the OWASP Top 10, which identifies the most critical web application security risks based on industry research and real-world attack trends. Vulnerabilities such as SQL Injection, Cross-Site Scripting (XSS), Broken Access Control, Security Misconfiguration, and Identification and Authentication Failures continue to affect thousands of web applications worldwide. As a result, security assessment tools that evaluate applications against OWASP recommendations have become valuable resources for improving software security and promoting secure coding practices (OWASP Foundation, 2021).

### 1.1.5 Existing Problems

Although numerous commercial and open-source vulnerability assessment tools are available, many existing solutions present several challenges for students, small organizations, and beginner security practitioners. Some commercial tools require expensive licensing fees, while certain open-source tools primarily focus on vulnerability detection without providing comprehensive risk assessment, centralized scan management, remediation guidance, or integrated learning resources. In addition, some tools generate highly technical reports that are difficult for inexperienced users to interpret, limiting their effectiveness as educational and decision-support platforms (PortSwigger, 2025; OWASP Foundation, 2025).

### 1.1.6 Our System

Therefore, this project proposes VulnScan, an Intelligent Web Application Vulnerability Assessment Platform with Risk Assessment and Remediation Knowledge Base. The proposed system is designed to automatically assess web applications for multiple security vulnerabilities, calculate risk levels based on standardized assessment methods, maintain detailed audit logs, and provide remediation recommendations through an integrated knowledge base. By combining automated vulnerability assessment with educational resources and centralized management features, VulnScan aims to improve both vulnerability detection and users' understanding of web application security while providing a scalable platform capable of supporting future vulnerability assessment modules.

## 1.2 Problem Statement

### 1.2.1 Complexity of Existing Tools

Web application vulnerability assessment plays a crucial role in identifying security weaknesses before they can be exploited by malicious attackers. Although numerous vulnerability assessment tools are available, many existing solutions are primarily designed for experienced cybersecurity professionals and penetration testers. Tools such as OWASP ZAP and Burp Suite provide comprehensive scanning capabilities but often require extensive security knowledge to configure scan settings, interpret scan results, and understand technical vulnerability reports (OWASP Foundation, 2025; PortSwigger, 2025). This steep learning curve makes these tools less suitable for beginner developers, students, and small organizations that lack dedicated cybersecurity expertise. Consequently, there is a need for a vulnerability assessment platform that offers an intuitive interface while maintaining effective vulnerability detection capabilities.

### 1.2.2 Lack of Integrated Risk Assessment and Remediation Guidance

Another limitation of many existing vulnerability assessment tools is their primary focus on vulnerability detection rather than vulnerability understanding and remediation. Most scanners generate technical reports that list detected vulnerabilities but provide limited explanation regarding the severity, business impact, secure coding practices, or recommended mitigation strategies. As a result, users may successfully identify vulnerabilities without fully understanding their associated risks or the appropriate remediation methods (NIST, 2020; OWASP Foundation, 2021). This limitation reduces the educational value of vulnerability assessment tools, particularly for software developers and students who require additional guidance to improve secure software development practices.

### 1.2.3 Limited Flexibility and Centralized Management

Furthermore, many vulnerability assessment tools offer limited flexibility in managing scanning configurations and supported vulnerability modules. As cybersecurity threats continue to evolve, new vulnerability detection capabilities must be introduced without significantly modifying the overall system architecture. Traditional implementations often tightly couple scan configurations with specific vulnerability detection functions, making future system enhancement more difficult. In addition, many lightweight vulnerability scanners lack centralized management features such as user administration, security audit logging, configurable scan profiles, and modular vulnerability management. These limitations reduce system maintainability, scalability, and suitability for long-term use.

### 1.2.4 Proposed Solution

To address the identified problems, this project proposes the development of VulnScan, an Intelligent Web Application Vulnerability Assessment Platform with Risk Assessment and Remediation Knowledge Base. The proposed system integrates automated vulnerability assessment, configurable scan profiles, risk assessment, centralized user and audit management, and a knowledge base that provides remediation recommendations and secure coding practices. The system adopts a modular architecture that supports future expansion of vulnerability detection capabilities while maintaining a user-friendly interface suitable for both beginner developers and cybersecurity practitioners.

## 1.3 Objectives of the Project

The main objective of this project is to develop VulnScan, an intelligent web application vulnerability assessment platform that assists developers in identifying web application security vulnerabilities, assessing security risks, and providing appropriate remediation guidance through an integrated knowledge base.

To achieve the main objective, the following specific objectives are proposed:

1. To design a web-based vulnerability assessment platform capable of automatically scanning web applications for common security vulnerabilities based on selected OWASP web security practices.
2. To implement a risk assessment mechanism that evaluates detected vulnerabilities according to their severity levels and presents the results in an understandable and structured manner.
3. To develop an integrated vulnerability knowledge base that provides detailed vulnerability descriptions, security impacts, remediation recommendations, secure coding practices, and reference materials to assist users in understanding and resolving identified security issues.
4. To design a centralized management system that supports user authentication, role-based access control, configurable scan profiles, vulnerability management, and comprehensive security audit logging.

## 1.4 Scope and Limitations

The scope of the proposed system defines the functionalities and boundaries of VulnScan. The project focuses on the development of an intelligent web-based vulnerability assessment platform that assists users in identifying common web application security vulnerabilities, assessing associated risks, and providing appropriate remediation guidance. The scope of the project is divided into functional scope and project limitations.

### 1.4.1 Functional Scope

The proposed system includes the following functionalities:

1. User Authentication and Role-Based Access Control
    a. The system supports secure user authentication using username and password.
    b. Two user roles are provided: Administrator and Developer.
    c. Administrators manage users, vulnerabilities, scan profiles, and system configurations, while Developers perform vulnerability assessments and view scan results.
2. Web Application Vulnerability Assessment
    a. The system allows users to initiate vulnerability scans by specifying a target web application URL.
    b. The scanner automatically crawls accessible web pages and performs vulnerability assessments based on the selected scan profile.
3. Configurable Scan Profiles
    a. Administrators can configure predefined scan profiles consisting of selected vulnerability modules and scanning depth.
    b. Developers select an appropriate scan profile before starting a vulnerability assessment.
4. Risk Assessment
    a. The system evaluates detected vulnerabilities according to predefined severity levels and presents the results through an organized dashboard.
    b. Risk assessment information assists users in prioritizing remediation efforts.

5. Vulnerability Knowledge Base
    a. The system provides comprehensive information for each supported vulnerability, including descriptions, security impacts, remediation recommendations, secure coding practices, and OWASP references (OWASP Foundation, 2021).
6. Security Audit Logging
    a. The system records significant security-related activities such as user authentication, scan execution, administrative actions, and report generation to improve accountability and traceability.

### 1.4.2 Project Limitations

The proposed system is developed within the constraints of an undergraduate Final Year Project. Therefore, several limitations are identified:

1. The system focuses on selected common web application vulnerabilities suitable for the project scope and does not aim to provide comprehensive penetration testing comparable to commercial security assessment platforms.
2. The system performs automated vulnerability assessment only and does not exploit or actively attack target systems beyond the implemented detection mechanisms.
3. Vulnerability detection modules are implemented by the system developer. Although administrators can manage vulnerability information and scan profiles, they cannot create new detection algorithms through the system interface.
4. Risk assessment is intended to assist decision-making and should not replace professional cybersecurity assessment or manual penetration testing performed by experienced security professionals.
5. The proposed system is designed primarily for educational purposes, research, and small-scale security assessment environments.

## 1.5 Significance of the Project

The development of VulnScan provides significant contributions to web application security by offering an intelligent vulnerability assessment platform that combines automated scanning, risk assessment, and security knowledge into a single integrated system. The project is expected to benefit multiple groups of users, including software developers, cybersecurity practitioners, students, educational institutions, and organizations that seek to improve the security of their web applications (NIST, 2020).

Firstly, the proposed system benefits software developers by enabling early identification of common web application security vulnerabilities during the software development process. By detecting security weaknesses before system deployment, developers can reduce potential security risks, improve software quality, and adopt secure coding practices. The integrated vulnerability knowledge base further assists developers in understanding the root causes of vulnerabilities and implementing appropriate remediation techniques.

Secondly, the project serves as a valuable educational platform for students and beginner cybersecurity practitioners. Unlike many existing vulnerability assessment tools that primarily focus on technical vulnerability detection, VulnScan integrates detailed vulnerability explanations, security impacts, remediation recommendations, secure coding practices, and OWASP references within the same platform. This allows users not only to identify security issues but also to understand the underlying security concepts, thereby supporting continuous learning in web application security.

Thirdly, the proposed system provides practical benefits to small and medium-sized organizations (SMEs) that may have limited cybersecurity resources. By offering a centralized platform for vulnerability assessment, configurable scan profiles, risk assessment, and security audit logging, the system assists organizations in monitoring the security posture of their web applications while improving accountability and security management practices.

Finally, this project contributes to the academic and research community by demonstrating the application of secure software engineering principles, vulnerability assessment methodologies, database normalization, modular system architecture, and role-based access control within a real-world cybersecurity application. The project also establishes a scalable system architecture that supports future expansion through additional vulnerability detection modules and enhanced security analysis capabilities.

Overall, the proposed system aims to improve web application security awareness while providing an effective, user-friendly, and scalable vulnerability assessment platform suitable for educational, research, and practical security assessment purposes.

## 1.6 Organisation of Report

This report is organised into six chapters, each focusing on a specific aspect of the proposed project.

- **Chapter 1** introduces the proposed project by presenting the project background, problem statement, objectives, scope and limitations, significance of the project, and the overall organisation of the report. This chapter establishes the motivation and direction of the proposed system.
- **Chapter 2** reviews the literature related to web application vulnerability assessment, cybersecurity concepts, and existing vulnerability assessment tools. Similar systems are analysed and compared to identify their strengths, limitations, and research gaps. The findings from the literature review provide the foundation for the proposed system design.
- **Chapter 3** describes the research methodology adopted throughout the project. It explains the project development methodology, system development process, software and hardware requirements, overall system architecture, database design, and implementation approach used to develop VulnScan.
- **Chapter 4** presents the implementation and development of the proposed system. It discusses the implementation of major system modules, including user authentication, vulnerability scanning, risk assessment, scan profile management, vulnerability management, audit logging, and the integrated knowledge base. Database implementation and user interface design are also presented in this chapter.

- **Chapter 5** evaluates the developed system through system testing and validation. The chapter presents the testing methodology, test cases, testing results, discussion of system performance, and an evaluation of whether the project objectives have been successfully achieved.
- **Chapter 6** concludes the report by summarising the overall project, highlighting the project's contributions and achievements, discussing its limitations, and proposing recommendations and future enhancements for further system improvement.

## 1.7 Chapter Summary and Evaluation

*This chapter introduced the proposed project by discussing the background of web application security and the importance of vulnerability assessment in protecting modern web applications. The problems associated with existing vulnerability assessment tools were identified, leading to the formulation of the project objectives and the proposed solution. The scope, limitations, and significance of the project were also presented to establish the boundaries and expected contributions of the proposed system. Finally, the overall organisation of the report was outlined to provide readers with an overview of the subsequent chapters. The next chapter reviews the relevant literature, existing technologies, and related systems that form the theoretical foundation of the proposed project.*

# Chapter 2

# **Literature Review**

# 2.0 Literature Review

This chapter reviews the literature related to web application security, vulnerability assessment methodologies, and technologies relevant to the development of the proposed system. The review begins by discussing fundamental concepts including web application security, vulnerability assessment, the OWASP Top 10, risk assessment, secure coding practices, and role-based access control. Subsequently, several existing vulnerability assessment tools are analysed and compared to identify their strengths and limitations. Finally, the research gaps identified from the literature review are discussed to justify the development of the proposed system, VulnScan.

## 2.1 Related Theories and Concepts

### 2.1.1 Web Application Security

Web application security refers to the practice of protecting web applications from unauthorized access, malicious attacks, data breaches, and other security threats throughout the software development lifecycle. Modern web applications frequently process sensitive information such as user credentials, financial records, healthcare information, and confidential business data, making them attractive targets for cybercriminals. Consequently, security must be incorporated into every phase of application development rather than being treated as a separate activity after deployment (OWASP Foundation, 2021).

Web application vulnerabilities commonly arise from insecure coding practices, improper authentication mechanisms, insufficient input validation, security misconfigurations, outdated software components, and inadequate access control. Attackers exploit these weaknesses to gain unauthorized access, execute malicious code, steal confidential information, or disrupt normal system operations. The increasing sophistication of cyberattacks has made web application security one of the most important domains within cybersecurity research and software engineering (OWASP Foundation, 2021).

To improve application security, organizations increasingly adopt secure software development practices, regular vulnerability assessments, security testing, and continuous monitoring throughout the software development lifecycle. International standards and security frameworks such as the OWASP Top 10 and the National Institute of Standards and Technology (NIST)

Cybersecurity Framework provide widely accepted guidelines for identifying and mitigating common web application security risks (OWASP Foundation, 2021).

### 2.1.2 Vulnerability Assessment

Vulnerability assessment is a systematic process of identifying, analysing, and evaluating security weaknesses within an information system or web application before they can be exploited by attackers. Unlike penetration testing, which actively attempts to exploit identified vulnerabilities, vulnerability assessment primarily focuses on discovering and documenting security weaknesses to support risk mitigation and secure software development (Scambray et al., 2011). The objective of vulnerability assessment is to provide developers and security practitioners with sufficient information to prioritize remediation activities and strengthen the overall security posture of an application.

Modern vulnerability assessment commonly combines automated scanning techniques with manual verification to improve detection accuracy. Automated vulnerability scanners significantly reduce the time required to identify common security weaknesses such as SQL Injection, Cross-Site Scripting (XSS), insecure HTTP headers, broken authentication, and security misconfigurations. However, automated tools may also generate false positives and false negatives, making human verification an important component of comprehensive security assessment (OWASP Foundation, 2021).

Vulnerability assessment has become an essential activity within the Secure Software Development Lifecycle (SSDLC). Performing security assessments during software development enables vulnerabilities to be identified earlier, reducing remediation costs and minimizing the likelihood of security incidents after deployment. Consequently, organizations increasingly integrate automated vulnerability scanning into continuous integration and continuous deployment (CI/CD) pipelines as part of DevSecOps practices (NIST, 2020).

### 2.1.3 OWASP Top 10

The Open Worldwide Application Security Project (OWASP) is an international non-profit organization dedicated to improving software security through open-source projects, educational resources, and industry best practices (OWASP Foundation, 2021). Among its most influential publications is the OWASP Top 10, which identifies the ten most critical web application security risks based on global industry data collected from security organizations and application assessments.

The OWASP Top 10 serves as an internationally recognized benchmark for secure web application development and is widely adopted by software developers, cybersecurity professionals, researchers, and educational institutions. Rather than introducing new vulnerabilities, the OWASP Top 10 prioritizes the most prevalent and impactful security risks affecting modern web applications, allowing organizations to focus their security efforts on the areas with the greatest potential impact (OWASP Foundation, 2021).

Since VulnScan is designed as a web application vulnerability assessment platform, the OWASP Top 10 provides the primary security framework for selecting and implementing vulnerability detection modules. In the initial version of the proposed system, VulnScan focuses on detecting selected high-priority vulnerabilities, including SQL Injection, Cross-Site Scripting (XSS), and missing HTTP Security Headers. These vulnerabilities were selected because they represent common security weaknesses that are suitable for automated detection while remaining achievable within the scope of an undergraduate Final Year Project.

**Table 2.1:** Selected OWASP Vulnerabilities Supported by VulnScan

| OWASP Category | Description | Supported in VulnScan |
|---|---|---|
| SQL Injection | Improper validation of user input allows malicious SQL statements to be executed. | Yes |
| Cross-Site Scripting (XSS) | Untrusted scripts are injected into web pages viewed by users. | Yes |
| Security Misconfiguration (HTTP Security Headers) | Missing or improperly configured HTTP security headers expose applications to additional security risks. | Yes |

### 2.1.4 Risk Assessment

Risk assessment is an essential component of vulnerability management because it enables organizations and developers to determine the severity and priority of identified security vulnerabilities. Simply detecting vulnerabilities is insufficient, as different vulnerabilities pose different levels of risk depending on their potential impact and likelihood of exploitation. Therefore, risk assessment provides a systematic approach for evaluating vulnerabilities and assisting decision-makers in prioritizing remediation efforts (FIRST, 2023).

One of the most widely adopted standards for vulnerability risk assessment is the Common Vulnerability Scoring System (CVSS), developed by the Forum of Incident Response and Security Teams (FIRST). CVSS provides a standardized numerical score ranging from 0.0 to 10.0, allowing vulnerabilities to be consistently evaluated and compared across different systems and organizations (FIRST, 2023). The scoring methodology considers several characteristics of a

vulnerability, including attack complexity, required privileges, user interaction, confidentiality impact, integrity impact, and availability impact.

CVSS classifies vulnerabilities into several severity levels, enabling organizations to prioritize remediation activities effectively. Vulnerabilities with higher scores require immediate attention because they present greater security risks, whereas lower-severity vulnerabilities may be addressed during routine maintenance. The standardized scoring system has become widely adopted by cybersecurity vendors, vulnerability scanners, and security researchers because it provides a common language for communicating security risks (Mell et al., 2007).

In the proposed VulnScan system, risk assessment is incorporated to help users understand the seriousness of detected vulnerabilities instead of merely presenting a list of findings. After vulnerabilities are identified, the system assigns a severity level and corresponding risk score, allowing users to prioritize remediation efforts more effectively. Presenting vulnerabilities according to their severity improves decision-making and enables developers to focus on addressing the most critical security issues first.

**Table 2.2:** CVSS Severity Ratings

| CVSS Score | Severity Level |
|---|---|
| 0.0 | None |
| 0.1 – 3.9 | Low |
| 4.0 – 6.9 | Medium |
| 7.0 – 8.9 | High |
| 9.0 – 10.0 | Critical |

### 2.1.5 Secure Coding Practices

Secure coding practices refer to software development techniques that minimize the introduction of security vulnerabilities during the implementation phase of the Software Development Life Cycle (SDLC). Rather than relying solely on vulnerability assessment after development, secure coding aims to prevent vulnerabilities from being introduced in the first place. This proactive approach has become a fundamental principle in modern software engineering because correcting security vulnerabilities during development is significantly less expensive than fixing them after deployment (Pressman & Maxim, 2020).

Many common web application vulnerabilities originate from poor programming practices such as insufficient input validation, improper authentication mechanisms, insecure session management, and inadequate access control. For example, SQL Injection attacks frequently occur when user input is directly incorporated into SQL queries without proper parameterization, while Cross-Site Scripting (XSS) vulnerabilities arise when untrusted user input is displayed without appropriate output encoding (OWASP Foundation, 2021).

To reduce these security risks, software developers are encouraged to adopt secure coding techniques including input validation, output encoding, parameterized SQL queries, secure password hashing, least privilege access control, proper error handling, and secure session management. These practices significantly reduce the attack surface of web applications while improving software reliability and maintainability (OWASP Foundation, 2021).

The proposed VulnScan system supports secure software development by not only identifying vulnerabilities but also providing remediation recommendations and secure coding practices through its integrated Knowledge Base. This enables developers to understand the root causes of detected vulnerabilities and apply appropriate mitigation techniques, thereby transforming vulnerability assessment into a valuable learning process.

### 2.1.6 Role-Based Access Control (RBAC)

Role-Based Access Control (RBAC) is an access control model that restricts system access based on the roles assigned to authenticated users. Instead of assigning permissions directly to individual users, permissions are associated with predefined roles, and users inherit permissions according to their assigned roles. This approach simplifies access management, improves security, and supports the principle of least privilege by ensuring users only possess the permissions necessary to perform their assigned responsibilities (Ferraiolo et al., 2001).

RBAC has become one of the most widely implemented access control mechanisms in modern information systems because it improves scalability and administrative efficiency. Organizations with large numbers of users can manage permissions more effectively by modifying role definitions rather than configuring permissions individually for each user. Furthermore, RBAC reduces the likelihood of unauthorized access resulting from excessive user privileges or administrative errors (NIST, 2025).

The proposed VulnScan system adopts the RBAC model by defining two user roles: Administrator and Developer. Administrators are responsible for managing user accounts, scan profiles, vulnerability information, and system configurations, while Developers primarily perform vulnerability assessments, review scan results, and access the integrated Knowledge Base. Separating responsibilities according to user roles enhances system security while providing a structured management framework suitable for both educational and organizational environments.

## 2.2 Study of Similar Existing Systems

### 2.2.1 Review of Similar Existing System

**A. OWASP ZAP (Zed Attack Proxy)**

OWASP Zed Attack Proxy (OWASP ZAP) is one of the most widely used open-source web application security testing tools developed by the Open Worldwide Application Security Project (OWASP). The tool is specifically designed to assist developers, security testers, and penetration testers in identifying security vulnerabilities within web applications during both development and testing phases (OWASP Foundation, 2025).

OWASP ZAP provides numerous security testing features, including automated vulnerability scanning, web crawling (spidering), passive and active vulnerability scanning, fuzzing, authentication testing, session management analysis, and API security testing. In addition, the tool supports scripting and plugin extensions, allowing experienced users to customize scanning behaviour according to different testing requirements (OWASP Foundation, 2025).

One of the major strengths of OWASP ZAP is that it is completely open source and freely available, making it highly accessible for educational institutions, students, and organizations with limited financial resources. Furthermore, its active community continuously maintains and updates vulnerability detection capabilities to address newly emerging security threats.

Despite its comprehensive functionality, OWASP ZAP presents several limitations, particularly for beginner users. The large number of configuration options, scanning parameters, and technical terminology may create a steep learning curve for individuals with limited cybersecurity experience. In addition, vulnerability reports generated by OWASP ZAP primarily focus on technical findings and provide limited educational guidance regarding secure coding practices or detailed remediation explanations. Consequently, users may successfully identify vulnerabilities without fully understanding how to resolve them effectively.

The proposed VulnScan system adopts several concepts inspired by OWASP ZAP, including automated vulnerability assessment, configurable scan profiles, and vulnerability detection. However, VulnScan additionally integrates risk assessment, a remediation knowledge base, role-based administration, and centralized security audit logging to improve usability and educational value, particularly for students and beginner developers.

**B. Burp Suite Community Edition**

Burp Suite Community Edition is a widely recognized web application security testing platform developed by PortSwigger. It is extensively used by cybersecurity professionals, penetration testers, and security researchers for discovering and analysing web application vulnerabilities. Burp Suite functions primarily as an intercepting proxy that enables users to inspect, modify, and replay HTTP requests and responses exchanged between web browsers and web servers (PortSwigger, 2025).

The Community Edition includes several important security testing tools, including the Proxy, Repeater, Intruder (limited functionality), Decoder, Comparer, and basic crawling capabilities. These tools enable users to manually investigate application behaviour, manipulate requests, and identify potential security weaknesses. Compared to the Professional Edition, however, the Community Edition does not provide a fully automated vulnerability scanner, requiring users to perform many testing activities manually (PortSwigger, 2025).

A significant advantage of Burp Suite is its powerful manual testing capabilities, making it one of the most trusted tools among professional penetration testers. Its intercepting proxy provides deep visibility into web application communication, enabling advanced security analysis beyond automated vulnerability scanning.

However, Burp Suite Community Edition is generally more suitable for experienced users who possess strong knowledge of HTTP protocols, web technologies, and penetration testing methodologies. The tool also places considerable emphasis on manual testing rather than beginner-friendly automation and educational support. Consequently, students and novice developers may experience difficulties understanding scan results and identifying appropriate remediation strategies.

Unlike Burp Suite Community Edition, VulnScan focuses primarily on providing automated vulnerability assessment combined with integrated risk evaluation, educational vulnerability explanations, remediation guidance, and user-friendly dashboards. The objective is not to replace professional penetration testing tools but rather to provide an accessible platform that improves both vulnerability detection and cybersecurity learning.

### C. Acunetix

Acunetix is a commercial web application vulnerability scanner designed to automate security testing for enterprise environments. The platform provides comprehensive vulnerability assessment capabilities, including website crawling, vulnerability scanning, authentication testing, API security assessment, and detailed security reporting. Acunetix supports the detection of thousands of known security vulnerabilities while integrating with software development workflows to facilitate continuous security testing (Invicti Security, 2025).

One of Acunetix's primary strengths is its high degree of automation. The platform automatically crawls websites, identifies potential attack surfaces, and performs extensive vulnerability analysis with minimal user intervention. In addition, Acunetix generates professional reports containing vulnerability descriptions, severity classifications, remediation recommendations, and compliance-related information, making it suitable for enterprise security management.

Despite these advantages, Acunetix is a commercial product requiring paid licensing, which may limit accessibility for students, educational institutions, and small organizations. Furthermore, while the platform provides detailed technical reports, its primary objective is professional vulnerability management rather than cybersecurity education. Consequently, users may still require additional learning resources to fully understand secure coding principles and vulnerability remediation techniques.

The proposed VulnScan system adopts several concepts from commercial vulnerability scanners, including automated assessment, structured reporting, and risk prioritization. However, VulnScan differentiates itself by integrating an educational knowledge base, configurable vulnerability modules, centralized user management, and security audit logging while maintaining accessibility for educational and research purposes.

### 2.2.2 Comparison of These Systems

Table 2.3 presents a comparison between three existing vulnerability assessment systems and the proposed VulnScan system. The comparison is based on several evaluation criteria that are relevant to the objectives of this project, including vulnerability assessment capability, risk assessment, remediation guidance, user management, scan profile configuration, security audit logging, educational support, and accessibility.

**Table 2.3:** Comparison of Existing Vulnerability Assessment Systems

| **Features** | **OWASP ZAP** | **Burp Suite Community** | **Acunetix** | **Proposed VulnScan** |
|---|---|---|---|---|
| Open Source | ✓ | ✓ (Community Edition) | ✗ | ✓ |
| Automated Vulnerability Scanning | ✓ | Limited | ✓ | ✓ |
| Manual Security Testing | ✓ | ✓ | Limited | ✗ |
| Risk Assessment | Limited | Limited | ✓ | ✓ |
| Vulnerability Knowledge Base | Limited | Limited | Limited | ✓ |
| Remediation Guidance | Basic | Basic | ✓ | ✓ |
| Configurable Scan Profiles | Limited | ✗ | ✓ | ✓ |
| Role-Based Access Control | ✗ | ✗ | ✓ | ✓ |

| | | | | |
|---|---|---|---|---|
| Security Audit Logging | ✗ | ✗ | Limited | ✓ |
| Educational Focus | Moderate | Low | Low | High |
| Suitable for Beginners | Moderate | Low | Moderate | High |
| Web-Based Interface | ✗ | ✗ | ✓ | ✓ |

From the comparison presented in Table 2.3, it can be observed that each existing vulnerability assessment tool possesses unique strengths. OWASP ZAP provides comprehensive automated vulnerability scanning and is widely adopted within the cybersecurity community because of its open-source nature and extensive security testing capabilities (OWASP Foundation, 2025). Burp Suite Community Edition is recognized for its powerful manual penetration testing features and remains one of the most popular tools among professional security testers (PortSwigger, 2025). Acunetix offers advanced commercial vulnerability management with extensive automation and enterprise reporting capabilities (Invicti Security, 2025).

Despite these advantages, none of the reviewed systems fully satisfies the objectives established for the proposed VulnScan platform. Existing solutions primarily emphasize vulnerability detection and professional security assessment rather than integrating educational resources, centralized administration, configurable vulnerability management, and security audit logging into a single platform. Furthermore, beginner developers and students may experience difficulties interpreting technical scan reports without additional guidance regarding vulnerability impacts and remediation techniques.

Consequently, the proposed VulnScan system aims to combine automated vulnerability assessment with integrated risk assessment, configurable scan profiles, centralized vulnerability management, role-based access control, security audit logging, and an educational knowledge base. By integrating these features within a single web-based platform, VulnScan seeks to provide

a more accessible and educational vulnerability assessment solution while maintaining sufficient functionality for practical web application security assessment.

### 2.2.3 Analysis of User Feedback, Adoption Rate, and Success Stories

**A. User Feedback**

User feedback collected from official documentation, cybersecurity communities, industry reviews, and academic studies indicates that existing vulnerability assessment tools generally provide comprehensive security testing capabilities but differ significantly in terms of usability and learning experience. OWASP ZAP is widely appreciated for being a free and open-source vulnerability assessment tool with extensive scanning capabilities, making it particularly popular among students, educational institutions, and organizations with limited financial resources (OWASP Foundation, 2025). Similarly, Burp Suite Community Edition is highly regarded by cybersecurity professionals due to its powerful manual testing capabilities and flexibility in analysing web application traffic (PortSwigger, 2025). However, both tools are frequently reported to present a relatively steep learning curve for beginner users because of their technical interfaces, complex configuration options, and detailed vulnerability reports.

Acunetix has also received positive feedback for its highly automated vulnerability scanning, professional reporting features, and enterprise-oriented workflow integration. Nevertheless, its commercial licensing model may reduce accessibility for students and small organizations that require affordable vulnerability assessment solutions (Invicti Security, 2025). Overall, user feedback suggests that while existing tools provide effective vulnerability detection, opportunities remain for improving usability, educational support, and user-friendly vulnerability interpretation.

**B. Adoption Rate**

The literature indicates that existing vulnerability assessment tools have achieved widespread adoption across industry, academia, and cybersecurity research. OWASP ZAP has become one of the most widely adopted open-source web application security testing tools due to its cost-free availability, continuous community development, and support for integration within DevSecOps and Continuous Integration/Continuous Deployment (CI/CD) environments (OWASP Foundation, 2025). Burp Suite is considered an industry-standard penetration testing platform and

is extensively used by professional penetration testers, security consultants, and bug bounty researchers worldwide (PortSwigger, 2025).

In contrast, Acunetix is primarily adopted by commercial organizations seeking enterprise-level vulnerability management and automated security assessment. Its advanced automation capabilities and comprehensive reporting have contributed to its widespread adoption within medium and large organizations despite its commercial licensing model (Invicti Security, 2025). The broad adoption of these systems demonstrates the increasing importance of automated vulnerability assessment in modern software development and cybersecurity operations.

### C. Success Stories

Numerous organizations have successfully incorporated vulnerability assessment tools into their software development and cybersecurity practices. OWASP ZAP is frequently used by universities, cybersecurity training providers, and open-source software communities to support web application security education and secure software development initiatives (OWASP Foundation, 2025). In addition, many organizations integrate OWASP ZAP into automated DevSecOps pipelines to continuously identify security vulnerabilities during software development.

Burp Suite has become one of the most commonly used tools among professional penetration testers and security researchers, particularly in vulnerability assessments, security audits, and bug bounty programmes. Meanwhile, Acunetix has demonstrated success within enterprise environments by enabling organizations to automate vulnerability management processes, improve regulatory compliance, and strengthen organizational cybersecurity posture through continuous security monitoring (Invicti Security, 2025).

The widespread adoption and successful implementation of these systems demonstrate the effectiveness of automated vulnerability assessment technologies. However, the literature also suggests that there remains an opportunity to develop a platform that combines automated vulnerability detection with educational support, integrated remediation guidance, centralized management, and user-friendly interfaces. These observations further justify the development of the proposed VulnScan system.

## 2.3 Research Gap

The literature review indicates that numerous vulnerability assessment tools have been developed to identify security weaknesses within web applications. Open-source tools such as OWASP ZAP provide comprehensive automated vulnerability scanning, while Burp Suite Community Edition offers powerful manual security testing capabilities. Commercial solutions such as Acunetix further extend vulnerability management through enterprise reporting and workflow integration. Although these systems perform effectively within their intended domains, several research and practical gaps remain.

Firstly, many existing vulnerability assessment tools primarily focus on identifying security vulnerabilities without providing sufficient educational support for users. Scan reports frequently contain technical terminology that may be difficult for students, beginner developers, and inexperienced security practitioners to interpret. As a result, users may identify vulnerabilities successfully but still lack sufficient understanding of their causes, impacts, and remediation techniques (OWASP Foundation, 2021).

Secondly, many existing tools separate vulnerability detection from security knowledge. Users often need to consult external documentation, OWASP Cheat Sheets, or additional learning resources to understand detected vulnerabilities and implement appropriate secure coding practices. This fragmented learning process reduces the overall effectiveness of vulnerability assessment, particularly in educational environments.

Thirdly, although several commercial vulnerability management platforms support centralized administration, these solutions are often associated with significant licensing costs that may limit accessibility for educational institutions and small organizations. Conversely, many open-source solutions provide extensive scanning functionality but lack integrated features such as configurable scan profiles, centralized vulnerability management, role-based administration, and comprehensive security audit logging.

Finally, many existing systems are designed primarily for experienced cybersecurity professionals. Their interfaces, configuration options, and technical reporting may present a significant learning curve for users with limited cybersecurity experience. Consequently, there remains a need for a user-friendly vulnerability assessment platform that combines effective vulnerability detection with educational guidance and centralized management capabilities.

To address these identified gaps, this project proposes VulnScan, an intelligent web application vulnerability assessment platform that integrates automated vulnerability assessment, configurable scan profiles, centralized user management, comprehensive audit logging, risk assessment, and an integrated vulnerability knowledge base within a single web-based application. By combining technical vulnerability assessment with educational support, the proposed system aims to improve both application security and cybersecurity learning outcomes.

## 2.4 Summary of Key Findings

The literature review conducted in this chapter has identified several key findings that establish the theoretical and technical foundation for the development of the proposed VulnScan system.

**A. Web Application Security**

The review highlights that web applications have become one of the primary targets of cyberattacks due to their widespread use in handling sensitive information. Consequently, implementing effective security measures throughout the software development lifecycle is essential to minimize security vulnerabilities and protect organizational assets (OWASP Foundation, 2021; NIST, 2020).

**B. Vulnerability Assessment**

The literature demonstrates that vulnerability assessment is a proactive security practice that enables organizations to identify and evaluate security weaknesses before they are exploited by attackers. Automated vulnerability assessment tools significantly improve the efficiency of vulnerability detection, while standardized risk assessment methodologies such as the Common Vulnerability Scoring System (CVSS) assist developers in prioritizing remediation activities according to the severity of identified vulnerabilities (FIRST, 2023).

**C. Existing Vulnerability Assessment Systems**

The review of OWASP ZAP, Burp Suite Community Edition, and Acunetix indicates that these tools possess strong vulnerability assessment capabilities and are widely adopted within the cybersecurity industry. Nevertheless, each tool exhibits certain limitations with respect to educational support, usability for beginners, centralized management, and integrated vulnerability remediation guidance.

### D. Research Gap

The comparative analysis reveals that existing vulnerability assessment systems primarily focus on vulnerability detection rather than combining vulnerability assessment with educational resources, configurable scan management, centralized administration, and comprehensive security audit logging. These findings indicate an opportunity to develop a more integrated vulnerability assessment platform that addresses both technical and educational requirements.

### E. Design Implications for VulnScan

Based on the literature review and comparative analysis, several design decisions were established for the proposed VulnScan system. The system will adopt a modular vulnerability architecture to support future scalability, implement configurable scan profiles to improve scanning flexibility, integrate role-based access control for secure user management, maintain comprehensive security audit logs to enhance accountability, and provide an integrated vulnerability knowledge base to improve users' understanding of identified security issues. These design decisions directly address the research gaps identified throughout this chapter and form the foundation of the system architecture presented in Chapter 3.

## 2.5 Chapter Summary and Evaluation

*This chapter reviewed the literature related to web application security and the technologies relevant to the development of the proposed VulnScan system. The discussion began with fundamental concepts including web application security, vulnerability assessment, the OWASP Top 10, risk assessment, secure coding practices, and role-based access control. Existing vulnerability assessment systems were then analysed and compared to identify their strengths and limitations. Based on the literature review, several research gaps were identified, providing a clear justification for the proposed system. Overall, the findings presented in this chapter establish the theoretical and technical foundation for VulnScan. The following chapter discusses the research methodology, system architecture, database design, and implementation approach adopted in developing the proposed system.*

Chapter 3

# Methodology and Requirements Analysis

# 3.0 Methodology and Requirements Analysis

This chapter presents the methodology and requirements analysis adopted for the development of the proposed VulnScan: Intelligent Web Application Vulnerability Assessment Platform. A systematic development methodology is essential to ensure that the project is completed in an organised, efficient, and adaptable manner while satisfying the objectives established in Chapter 1.

The chapter begins by discussing the software development methodology selected for the project and provides a justification for its adoption. This is followed by an explanation of the techniques used to gather and document system requirements. The identified requirements are then analysed and classified into functional and non-functional requirements to establish a clear understanding of the expected behaviour and quality attributes of the proposed system.

Finally, the chapter presents the prioritisation and analysis of system requirements to ensure that the proposed solution aligns with stakeholder needs, project scope, and technical constraints. The outcomes of this chapter serve as the foundation for the system design presented in Chapter 4 and guide the subsequent implementation of VulnScan.

## 3.1 Project Methodology

The success of a software development project depends not only on technical implementation but also on the selection of an appropriate development methodology. A suitable methodology provides a structured framework that guides project planning, requirements analysis, system design, implementation, testing, and evaluation while ensuring that project objectives are achieved within the available time and resource constraints (Pressman & Maxim, 2020).

For the development of **VulnScan**, the **Agile Software Development Methodology** was selected. Agile is an iterative and incremental software development methodology that emphasizes continuous planning, collaboration, adaptability, and regular delivery of functional software. Unlike traditional linear methodologies such as the Waterfall model, Agile allows project requirements and system functionalities to evolve throughout the development lifecycle based on continuous evaluation and feedback (Sommerville, 2016).

The selection of Agile is particularly suitable for the development of VulnScan because cybersecurity applications frequently require continuous refinement as new security requirements, vulnerability detection techniques, and implementation challenges emerge during development. The modular nature of the proposed system also aligns well with Agile's iterative philosophy, allowing individual system modules to be designed, implemented, tested, and improved independently before being integrated into the complete system.

Furthermore, the project consists of several interconnected modules, including user authentication, vulnerability management, configurable scan profiles, web crawling, vulnerability detection, risk assessment, audit logging, and the remediation knowledge base. Developing these modules incrementally reduces development complexity while enabling each component to be evaluated and improved before proceeding to subsequent iterations. This approach minimizes implementation risks and allows design improvements to be incorporated without significantly affecting completed components.

Another important advantage of Agile is its flexibility in accommodating changes throughout the project lifecycle. During the design phase of VulnScan, several architectural improvements were introduced through iterative discussions and evaluations. For example, the original vulnerability configuration mechanism was redesigned into a more scalable **ProfileModule** architecture, enabling administrators to dynamically configure vulnerability modules without modifying the underlying database structure. Similarly, the database schema underwent several refinement cycles to achieve Third Normal Form (3NF), improve scalability, and eliminate data redundancy. These iterative refinements demonstrate the practical benefits of Agile in supporting continuous improvement while maintaining a stable and maintainable system architecture.

In addition to improving software quality, Agile promotes continuous verification of project deliverables. Instead of postponing testing until the end of development, each completed module is reviewed, validated, and refined before progressing to the next development stage. This iterative verification process reduces the likelihood of major design flaws being discovered late in the project while improving overall software reliability and maintainability.

Based on these considerations, Agile Software Development provides an appropriate methodological framework for the development of VulnScan. Its iterative, flexible, and modular approach supports the project's technical objectives while facilitating continuous improvement, efficient risk management, and systematic software development throughout the project lifecycle.

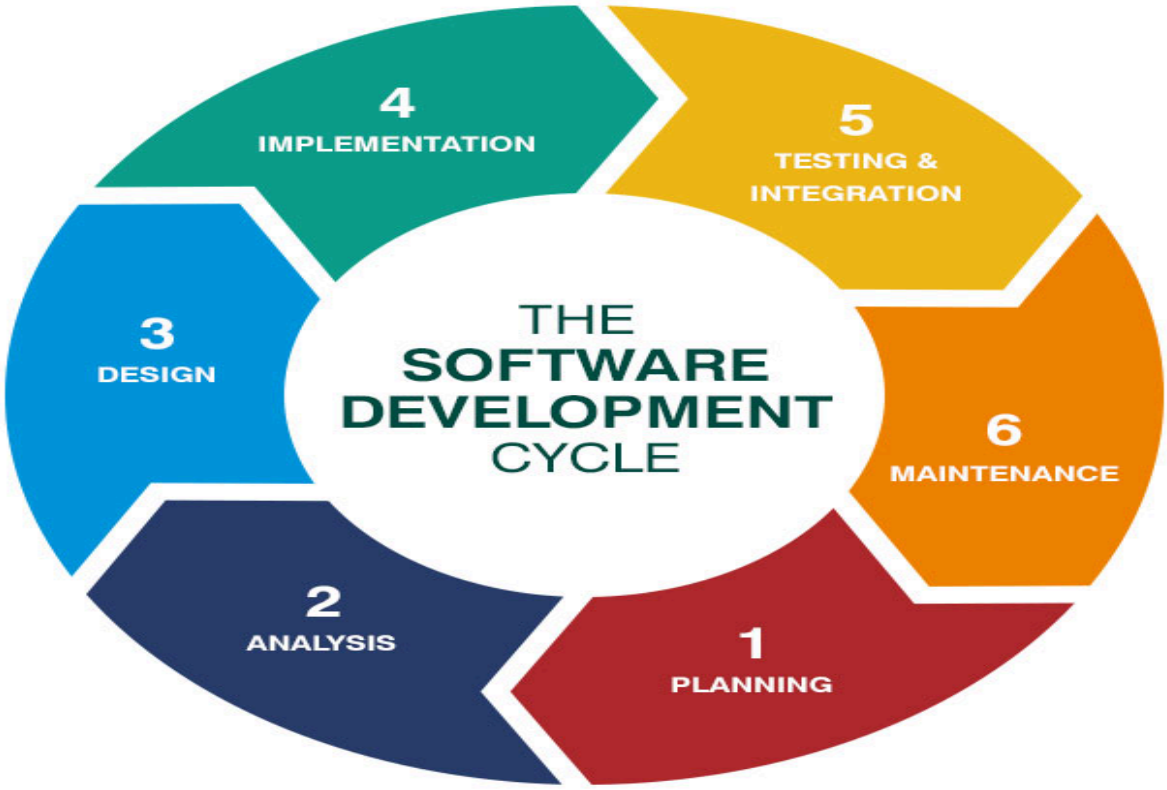



**Figure 3.1:** Agile Software Development Lifecycle adopted in the development of VulnScan.

### 3.1.1 Planning Phase

The Planning Phase is the first stage of the Agile software development methodology, where the objectives, project scope, and development strategy for VulnScan were established. During this phase, the problems addressed by the proposed system were identified through literature review, analysis of existing vulnerability assessment tools, and discussions with the project supervisor. The findings indicated that many existing tools are either too complex for beginner developers or lack integrated educational support, creating the motivation for developing VulnScan.

After identifying the problem, the project objectives, system scope, and expected deliverables were defined. The proposed system aims to provide automated web vulnerability assessment, configurable scan profiles, a vulnerability knowledge base, and security audit logging. To ensure the project remains achievable within the allocated development period, the initial implementation focuses on detecting SQL Injection (SQLi), Cross-Site Scripting (XSS), and HTTP Security Header vulnerabilities, while allowing future vulnerability modules to be incorporated through its modular architecture.

The Planning Phase also involved selecting suitable development technologies. Python was chosen as the programming language due to its extensive cybersecurity libraries, while Flask was selected as the backend framework because of its lightweight and modular architecture. Microsoft

SQL Server was selected as the database management system, with SQLAlchemy serving as the Object Relational Mapping (ORM) framework to simplify database interaction. The completion of this phase established a clear roadmap for the subsequent analysis, design, implementation, and testing phases.

### 3.1.2 Analysis Phase

The Analysis Phase focuses on identifying and analysing the functional and non-functional requirements of VulnScan. During this phase, information gathered from the literature review, existing systems, project objectives, and discussions with the project supervisor was analysed to determine the system requirements. The analysis ensured that the proposed system addresses the limitations identified in existing vulnerability assessment platforms while remaining aligned with the project objectives.

The functional requirements were identified based on the responsibilities of the two system users: Administrator and Developer. Developers are responsible for performing vulnerability scans, viewing scan results, and accessing remediation guidance, whereas Administrators manage user accounts, vulnerability definitions, scan profiles, and security audit logs. These requirements were later translated into the system modules developed for VulnScan.

In addition to functional requirements, several non-functional requirements were identified, including security, scalability, maintainability, performance, and usability. Throughout this phase, the proposed system architecture and database design were refined through multiple iterations to improve scalability and eliminate redundancy. For example, the introduction of the ProfileModule architecture enables vulnerability modules to be configured dynamically, while the database was normalised to Third Normal Form (3NF) to improve maintainability. The outputs of this phase provided the foundation for the system design presented in the next stage.

### 3.1.3 Design Phases

The Design Phase focuses on transforming the analysed system requirements into a structured system design that serves as the blueprint for implementation. During this phase, the overall architecture, database structure, system modules, and application workflow of VulnScan were designed based on the functional and non-functional requirements identified in the previous phase.

To ensure the system is maintainable and scalable, VulnScan adopts a layered architecture consisting of the Presentation Layer, REST API Layer, Business Logic Layer, Scanner Engine, and Database Layer. This architecture promotes separation of concerns by assigning specific responsibilities to each layer, making the system easier to develop, test, and maintain. In addition, the scanner engine was designed using a modular approach, allowing new vulnerability detection modules to be added in future without significantly modifying the existing system.

The database was designed using Third Normal Form (3NF) to minimise data redundancy and maintain data integrity. The final database consists of eight entities: **User, Scan, ScanProfile, ProfileModule, Vulnerability, DiscoveredPage, Finding, and SecurityAuditLog**. The entity relationships were carefully designed to support configurable scan profiles, modular vulnerability management, scan history, and comprehensive audit logging. The completed system design provided a clear blueprint for the implementation phase and is presented in detail in Chapter 4.

### 3.1.4 Implementation Phase

The Implementation Phase focuses on translating the approved system design into a functional software application. During this phase, the modules designed in the previous phase were developed incrementally following the Agile methodology, allowing each component to be implemented, tested, and refined before proceeding to the next module.

The development of VulnScan began with the implementation of the database, followed by the backend services using the Flask framework and SQLAlchemy ORM. Core functionalities such as user authentication, vulnerability management, scan profile management, web crawling, vulnerability detection, and security audit logging were implemented as independent modules. This modular development approach improves maintainability while allowing new functionalities to be added with minimal impact on existing components.

Throughout the implementation process, each completed module was verified before integration with other system components. This iterative development approach enabled issues to be identified and resolved early while ensuring that the implemented system remained consistent with the requirements and design established in the earlier phases. The detailed implementation of each module is presented in Chapter 4.

### 3.1.5 Testing and Integration

The Testing and Integration Phase ensures that the developed modules function correctly both individually and as an integrated system. In Agile software development, testing is performed continuously throughout the development process rather than being postponed until the end of the project. This approach enables defects to be identified and corrected early, reducing the likelihood of major issues during system integration (Pressman & Maxim, 2020).

For the VulnScan project, each module is tested individually before being integrated with other system components. Core functionalities, including user authentication, vulnerability management, scan profile management, web crawling, vulnerability detection, and security audit logging, are verified to ensure they operate according to the specified functional requirements. After individual testing, the modules are integrated and tested collectively to ensure that data flows correctly between the frontend, backend services, scanner engine, and database.

The testing process also verifies that the implemented system satisfies the functional and non-functional requirements identified during the Analysis Phase. Any issues discovered during

testing are corrected before the system proceeds to the next development iteration. This iterative testing and integration approach improves software quality, enhances system reliability, and ensures that VulnScan performs as expected before deployment.

### 3.1.6 Maintenance

The Maintenance Phase is the final stage of the Agile software development methodology. It focuses on maintaining and improving the system after implementation by correcting defects, enhancing existing functionalities, and adapting the system to new user requirements or technological changes (Sommerville, 2016). Although maintenance activities primarily occur after the system has been deployed, they are considered an essential part of the software development lifecycle to ensure the long-term reliability and effectiveness of the application.

For the VulnScan project, the modular system architecture adopted during the Design Phase simplifies future maintenance and enhancement activities. New vulnerability detection modules, such as Cross-Site Request Forgery (CSRF), XML External Entity (XXE), Server-Side Request Forgery (SSRF), and Command Injection, can be incorporated without requiring major modifications to the existing database structure or software architecture. Similarly, improvements to scan profiles, vulnerability definitions, and security recommendations can be introduced while maintaining compatibility with the existing system.

In addition to feature enhancements, future maintenance activities include fixing software defects, improving system performance, strengthening security mechanisms, and updating the vulnerability knowledge base to reflect newly discovered security threats. These continuous improvements ensure that VulnScan remains reliable, maintainable, and capable of adapting to the evolving cybersecurity landscape.

## 3.2 Requirement Gathering

Requirement gathering is a fundamental activity in software development that aims to identify and document the needs and expectations of stakeholders before system implementation begins. A well-defined set of requirements provides clear guidance for system design and development while ensuring that the completed system fulfils its intended objectives (Sommerville, 2016). For the development of VulnScan, the system requirements were primarily gathered through the analysis of existing vulnerability assessment systems and the identification of functional and non-functional requirements based on the project objectives established in Chapter 1.

The gathered requirements serve as the foundation for the design and implementation of VulnScan. They define the core functionalities that the system must provide, as well as the quality attributes required to ensure that the proposed platform is secure, reliable, scalable, and maintainable. The following sections discuss the existing system analysis and the identified functional and non-functional requirements.

### 3.2.1 Existing System Analysis

Existing system analysis was conducted to understand the capabilities, strengths, and limitations of current web application vulnerability assessment tools. This process provided valuable insights into common features adopted by existing solutions and helped identify opportunities for improvement in the proposed VulnScan platform.

The analysis focused on three widely recognised vulnerability assessment systems: OWASP Zed Attack Proxy (OWASP ZAP), Burp Suite Community Edition, and Acunetix. These systems were selected because they represent different categories of vulnerability assessment tools, including open-source, professional penetration testing, and commercial enterprise solutions. The comparative study conducted in Chapter 2 revealed that although these tools provide comprehensive vulnerability detection capabilities, they generally focus on security professionals and often require considerable technical knowledge to operate effectively.

Based on the findings from the existing system analysis, several requirements for VulnScan were identified. The proposed system should provide automated vulnerability scanning, configurable scan profiles, modular vulnerability detection, integrated vulnerability knowledge, security audit logging, and a user-friendly interface suitable for developers with varying levels of cybersecurity expertise. These identified requirements formed the basis for defining the functional and non-functional requirements discussed in the following sections.

### 3.2.2 Functional Requirements

Functional requirements define the services and functionalities that the proposed system must provide to satisfy user requirements and achieve the project objectives. These requirements describe the interactions between users and the system and specify the expected behaviour of VulnScan. Based on the existing system analysis and the project objectives, the functional requirements of VulnScan are summarised in Table 3.1.

**Table 3.1:** Functional Requirements of VulnScan

| Requirement ID | Functional Requirement | Description |
|---|---|---|
| FR-01 | User Authentication | The system shall allow users to register, log in securely, and log out using authenticated credentials. |
| FR-02 | Role-Based Access Control | The system shall differentiate access privileges between Administrators and Developers based on user roles. |
| FR-03 | Vulnerability Management | The system shall allow administrators to create, update, enable, disable, and maintain vulnerability definitions and remediation information. |
| FR-04 | Scan Profile Management | The system shall allow administrators to create and manage configurable scan profiles consisting of selected vulnerability modules. |
| FR-05 | Web Vulnerability Scanning | The system shall allow developers to initiate vulnerability scans by providing a target website URL and selecting a scan profile. |
| FR-06 | Web Crawling | The system shall automatically discover and analyse web pages within the specified scan scope before executing vulnerability detection modules. |

| | | |
|---|---|---|
| FR-07 | Vulnerability Detection | The system shall detect supported vulnerabilities, including SQL Injection (SQLi), Cross-Site Scripting (XSS), and HTTP Security Header misconfigurations. |
| FR-08 | Scan Result Management | The system shall store, display, and manage scan results, including detected vulnerabilities, affected pages, risk scores, and supporting evidence. |
| FR-09 | Vulnerability Knowledge Base | The system shall provide descriptions, impacts, secure coding practices, recommendations, and reference links for each supported vulnerability. |
| FR-10 | Security Audit Logging | The system shall record significant user and system activities, including login attempts, administrative actions, and scan operations, for auditing purposes. |

### 3.2.3 Non-Functional Requirements

Non-functional requirements specify the quality attributes and operational constraints of the proposed system. Unlike functional requirements, which describe the services provided by the system, non-functional requirements define how the system should perform and the standards it must satisfy. These requirements ensure that VulnScan is secure, reliable, maintainable, scalable, and capable of providing a satisfactory user experience. The identified non-functional requirements are summarised in Table 3.2.

**Table 3.2:** Non-Functional Requirements of VulnScan

| Requirement ID | Non-Functional Requirement | Description |
|---|---|---|
| NFR-01 | Security | The system shall protect user credentials through password hashing, enforce role-based access control, and record significant system activities using security audit logs. |

| | | |
|---|---|---|
| NFR-02 | Performance | The system shall perform vulnerability scans efficiently and respond to user requests within an acceptable time under normal operating conditions. |
| NFR-03 | Reliability | The system shall maintain consistent operation and preserve scan data accurately throughout the scanning process. |
| NFR-04 | Scalability | The system shall support the addition of new vulnerability detection modules and scan profiles without requiring major modifications to the existing architecture or database. |
| NFR-05 | Maintainability | The system shall adopt a modular software architecture and a normalized database structure to simplify future maintenance and enhancements. |
| NFR-06 | Usability | The system shall provide a user-friendly interface that enables developers to perform vulnerability scans and interpret scan results with minimal technical complexity. |
| NFR-07 | Compatibility | The system shall operate on modern web browsers and support integration with Microsoft SQL Server through the Flask backend. |
| NFR-08 | Availability | The system shall remain accessible whenever required during normal operation and recover gracefully from unexpected failures where possible. |

## 3.3 Requirement Analysis

Requirements analysis is performed to evaluate the identified functional and non-functional requirements and ensure that they are complete, feasible, and aligned with the objectives of the proposed VulnScan system. This process also assists in determining the implementation priority of each requirement while identifying any dependencies or potential conflicts between system functionalities. The analysed requirements provide a clear foundation for the system design presented in Chapter 4.

### 3.3.1 Analysis and Prioritization of Requirements

After the system requirements were identified, they were analysed and prioritised according to their importance to the successful operation of VulnScan. Core functionalities required for the basic operation of the system were assigned the highest priority, while supporting features were given lower priority. This prioritisation ensures that the essential system modules are developed first before additional functionalities are implemented.

The highest priority requirements include user authentication, role-based access control, vulnerability scanning, web crawling, scan result management, and vulnerability detection, as these modules form the core functionality of VulnScan. Supporting features such as the vulnerability knowledge base, scan profile management, and security audit logging are also important but are implemented after the core scanning functions have been established. This prioritisation enables the project to follow an iterative Agile development approach while ensuring that the most critical functionalities are completed first.

**Table 3.3:** Requirement Prioritization

| Priority | Requirements |
|---|---|
| High | User Authentication, Role-Based Access Control, Web Crawling, Vulnerability Detection, Scan Result Management |
| Medium | Scan Profile Management, Vulnerability Management, Security Audit Logging |
| Low | Vulnerability Knowledge Base, Future Vulnerability Modules |

### 3.3.2 Identification of Dependencies and Conflicts among Requirements

The identified system requirements were analysed to determine their dependencies and identify any potential conflicts before implementation. Understanding these relationships helps ensure that the system modules are developed in a logical sequence while reducing implementation risks.

Several functional requirements depend on the successful completion of other modules. For example, users must be authenticated before performing vulnerability scans, and scan results can only be generated after the web crawling and vulnerability detection processes have been completed. Similarly, security audit logging depends on user authentication and system activities to record meaningful events throughout the application's operation.

No significant conflicts were identified among the system requirements. However, certain non-functional requirements require careful consideration during development. For instance, increasing the depth of web crawling may improve vulnerability detection coverage but can also increase scanning time and system resource consumption. Therefore, configurable scan profiles are introduced to allow users to balance scanning accuracy and performance according to their requirements.

## 3.4 Summary

*This chapter presented the methodology and requirements analysis adopted for the development of VulnScan. The Agile software development methodology was selected to support iterative planning, design, implementation, testing, and continuous improvement throughout the project. The chapter also discussed the requirement gathering process, identified the functional and non-functional requirements of the proposed system, and analysed their priorities, dependencies, and potential conflicts.*

*The outcomes of this chapter establish a clear understanding of the system requirements and development approach. These findings provide the foundation for the system design presented in Chapter 4, where the software architecture, database design, user interface, and detailed system components of VulnScan are discussed*

Chapter 4

# System Design

# 4.0 System Design

Before designing the proposed system, the functional and non-functional requirements identified in Chapter 3 were analysed to ensure that the system architecture meets the project's objectives. This chapter presents the overall system design of VulnScan, including the architecture design, database design, user interface design, and detailed design of the major system modules. The proposed design emphasises modularity, maintainability, scalability, and security to support efficient vulnerability assessment while allowing future enhancements with minimal modifications.

The system design serves as the blueprint for the implementation of VulnScan. It illustrates how the major system components interact with one another, how data is organised within the database, how users interact with the system through the user interface, and how each module contributes to the overall functionality of the application. By establishing a clear and structured design, the proposed system aims to satisfy the functional and non-functional requirements identified in the previous chapter while supporting future system expansion.

## 4.1 Architecture Design

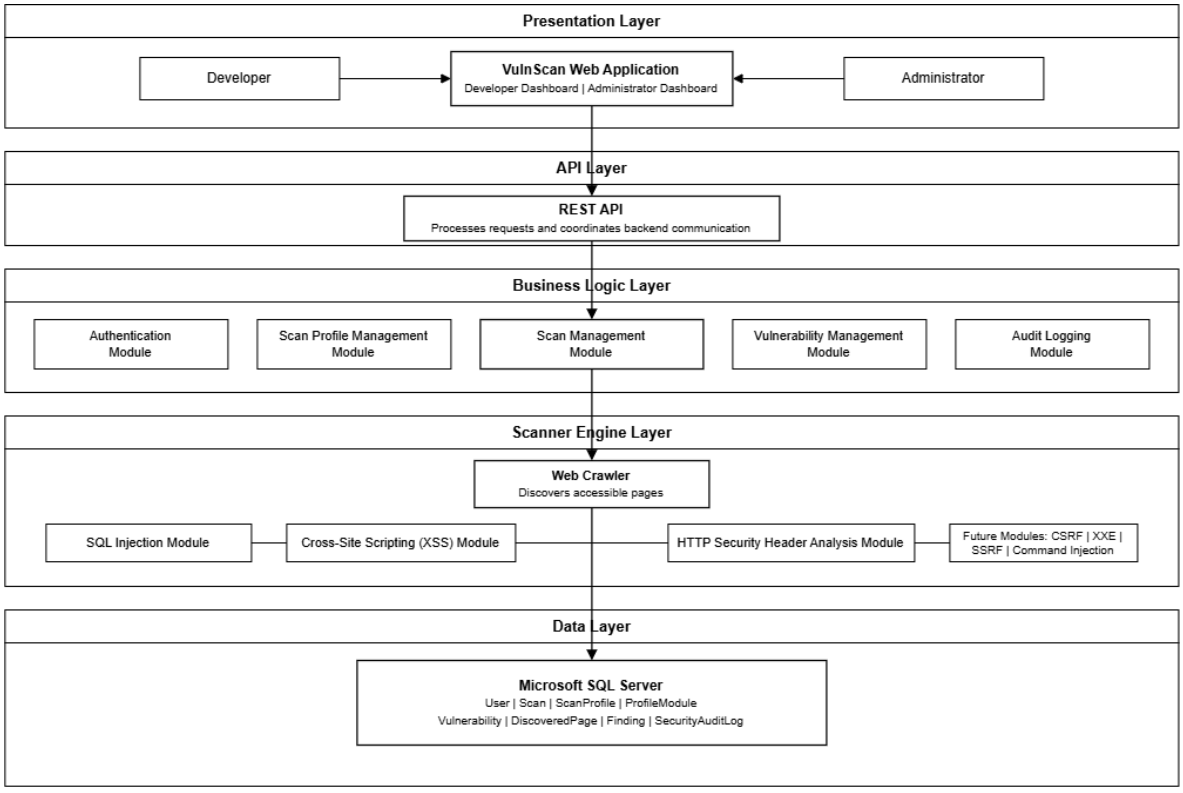



**Figure 4.1:** Overall System Architecture of VulnScan

The overall architecture of VulnScan is designed using a **layered architecture** to separate system responsibilities into distinct layers, thereby improving maintainability, scalability, and modularity. As illustrated in **Figure 4.1**, the system consists of five primary layers: the **Presentation Layer**, **API Layer**, **Business Logic Layer**, **Scanner Engine Layer**, and **Data Layer**. Each layer performs a specific function while interacting with adjacent layers through well-defined interfaces, reducing coupling between system components and simplifying future maintenance (Pressman & Maxim, 2020).

The **Presentation Layer** provides separate interfaces for Developers and Administrators through a web application. Developers are able to initiate vulnerability scans, review scan results, and manage scan profiles, whereas Administrators perform system management tasks such as user administration, vulnerability management, and monitoring system activities. User requests are forwarded to the **API Layer**, where the Flask REST API processes incoming requests and coordinates communication between the user interface and the backend services.

The **Business Logic Layer** contains the core application modules responsible for implementing the business rules of the system. These modules include Authentication, Scan Management, Scan Profile Management, Vulnerability Management, and Audit Logging. The Business Logic Layer validates user requests, applies access control, manages application data, and coordinates interactions with the Scanner Engine.

The **Scanner Engine Layer** is responsible for performing vulnerability assessments on target web applications. It consists of a Web Crawler and multiple independent vulnerability detection modules, including SQL Injection, Cross-Site Scripting (XSS), and HTTP Security Header analysis. The modular design enables additional vulnerability detection modules, such as Cross-Site Request Forgery (CSRF), XML External Entity (XXE), Server-Side Request Forgery (SSRF), and Command Injection, to be incorporated in future versions without requiring significant modifications to the overall system architecture.

The **Data Layer** utilises Microsoft SQL Server to manage application data, including user accounts, scan information, vulnerability definitions, scan profiles, findings, discovered pages, and security audit logs. This layered architecture promotes separation of concerns, facilitates future system enhancements, and ensures that each system component can be developed, tested, and maintained independently.

## 4.2 Database Design

The database is designed to provide a structured and reliable repository for managing user information, scan configurations, vulnerability definitions, scan results, and system activities. Microsoft SQL Server was selected as the database management system because of its robustness, transactional reliability, advanced security features, and seamless integration with SQLAlchemy. These capabilities make it suitable for managing the relational data required by VulnScan while supporting future scalability and maintenance.

The database design follows the principles of relational database modelling and has been normalised to Third Normal Form (3NF) to minimise data redundancy and maintain data consistency. Related data are separated into individual entities, with relationships established through primary and foreign keys to preserve referential integrity. This approach improves data organisation, simplifies maintenance, and reduces the risk of update anomalies.

As illustrated in Figure 4.2, the VulnScan database consists of eight entities: User, Scan, ScanProfile, ProfileModule, Vulnerability, DiscoveredPage, Finding, and SecurityAuditLog. Each entity is responsible for storing a specific category of information required by the system. The relationships among these entities enable the system to manage vulnerability scanning activities, configurable scan profiles, modular vulnerability definitions, detected findings, discovered web pages, and security audit records while maintaining consistency across the database.

The User entity manages user accounts, authentication information, and user roles, providing the foundation for role-based access control. The Scan entity stores information relating to each vulnerability assessment session and acts as the central entity connecting scan profiles, discovered pages, and detected findings. During execution, the Web Crawler records discovered web pages in the DiscoveredPage entity, while identified vulnerabilities are stored within the Finding entity and linked to the corresponding vulnerability definitions maintained in the Vulnerability entity.

To improve flexibility and future scalability, VulnScan adopts a configurable scan profile mechanism through the ScanProfile and ProfileModule entities. Rather than hardcoding supported vulnerabilities within the application, administrators can configure which vulnerability detection modules belong to each scan profile. This modular approach allows future detection modules, such as Cross-Site Request Forgery (CSRF), XML External Entity (XXE), Server-Side Request Forgery (SSRF), and Command Injection, to be incorporated without redesigning the database structure.

Finally, the SecurityAuditLog entity records significant user and system activities throughout the application. Maintaining audit records enhances accountability, supports security monitoring, and facilitates future forensic analysis by preserving a history of important events performed within the system.

**Table 4.1:** Description of Database Entities

| **Entity** | **Purpose** |
|---|---|
| User | Stores user account information, authentication details, and role assignments. |
| Scan | Records each vulnerability scanning session performed by users. |
| ScanProfile | Stores reusable scanning configurations. |
| ProfileModule | Defines the vulnerability detection modules associated with each scan profile. |
| Vulnerability | Maintains the vulnerability knowledge base, including descriptions and remediation guidance. |
| DiscoveredPage | Stores web pages identified during the crawling process. |
| Finding | Records vulnerabilities detected during each scan together with their severity and supporting evidence. |
| SecurityAuditLog | Records important user and system activities for auditing and security monitoring. |

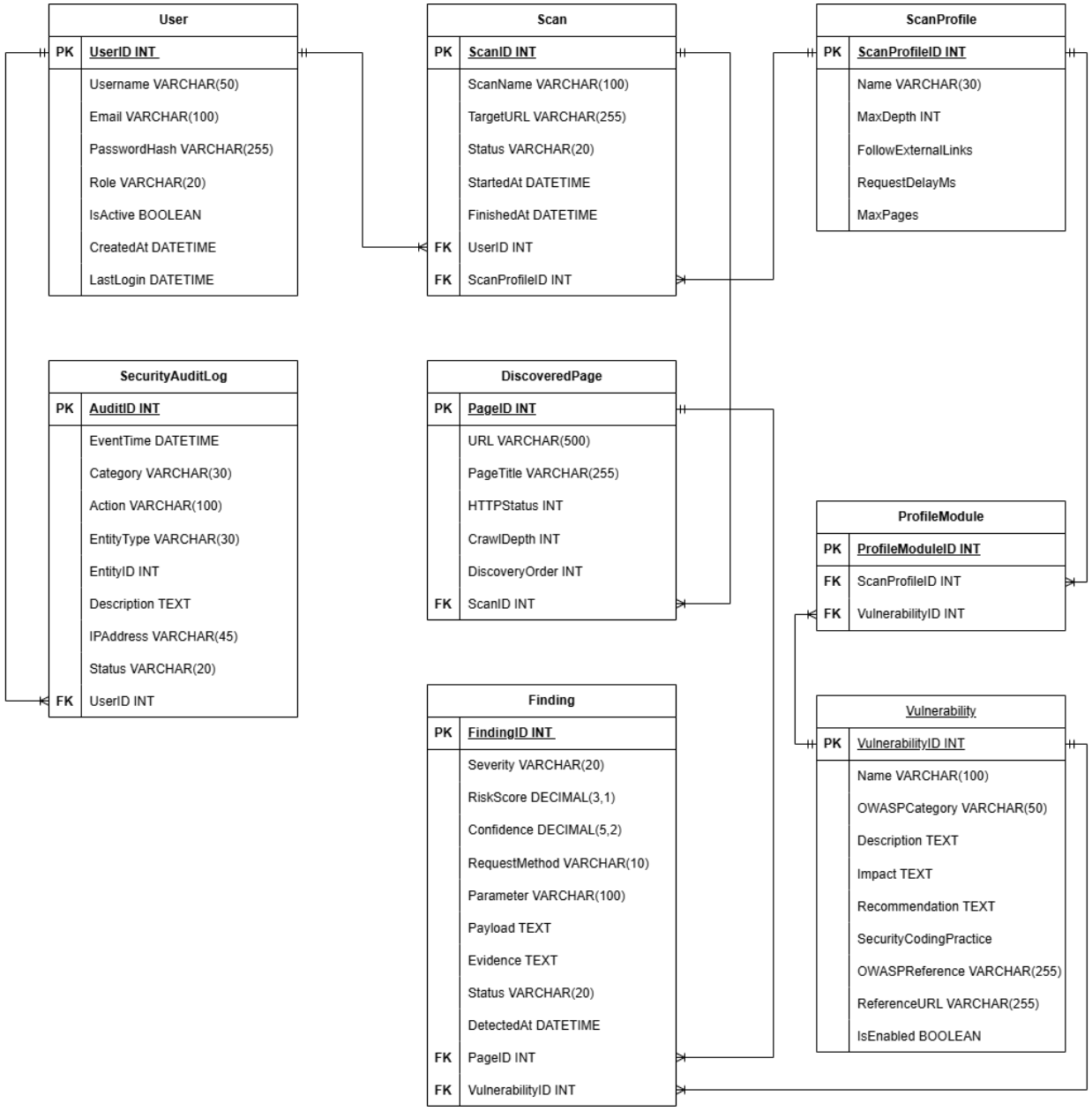



**Figure 4.2:** Entity Relationship Diagram (ERD) of VulnScan

## 4.3 User Interface Design

The user interface of VulnScan is designed to provide a simple, consistent, and intuitive experience for both Developers and Administrators. The interface emphasises usability by presenting system functions in a clear and organised manner, allowing users to perform vulnerability scanning and system administration with minimal complexity. Consistent layouts, navigation menus, and interface components are adopted throughout the system to improve user experience and reduce the learning curve.

The system provides separate interfaces based on user roles. After successful authentication, Developers are redirected to the Developer Dashboard, where they can manage scan profiles, initiate vulnerability scans, monitor scan progress, and review previous scan results. Administrators are redirected to the Administrator Dashboard, which provides additional functions such as user management, vulnerability management, scan profile configuration, and security audit log monitoring. This separation ensures that users only have access to functions relevant to their assigned roles.

The Login Page serves as the entry point to the system by allowing registered users to authenticate using their credentials. Upon successful login, users are redirected to the appropriate dashboard according to their assigned role. If authentication fails, appropriate error messages are displayed to guide users while preventing unauthorised access.

The Scan Management Page enables developers to configure and execute vulnerability scans by specifying the target URL and selecting a predefined scan profile. After the scan is completed, the Scan Result Page presents the detected vulnerabilities together with their severity level, affected pages, supporting evidence, and recommended remediation. The Administrator Dashboard provides management functions for maintaining users, vulnerabilities, scan profiles, and security audit logs, allowing administrators to oversee the overall operation of the system.

Overall, the user interface design focuses on usability, consistency, and accessibility to ensure that both developers and administrators can efficiently perform their respective tasks while interacting with VulnScan.

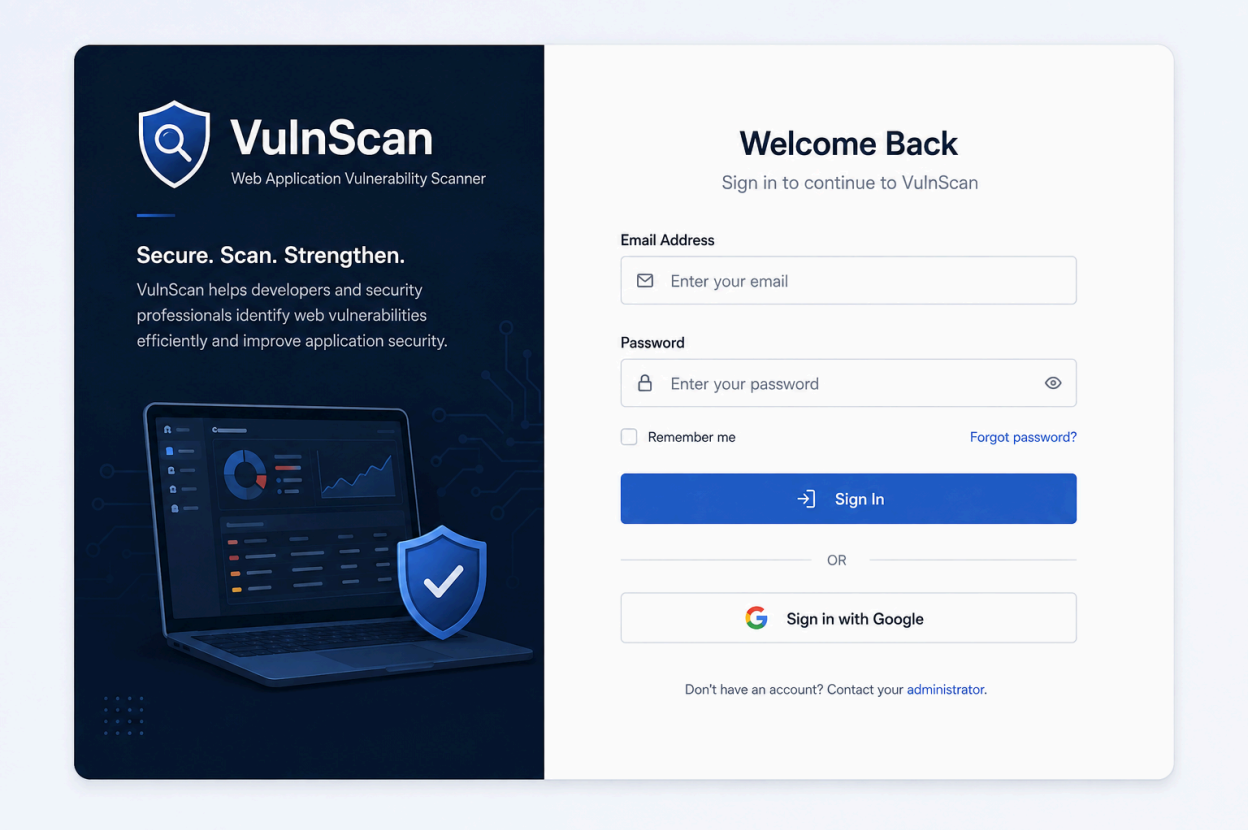



**Figure 4.3:** Login Page User Interface

**Purpose:**
This interface serves as the entry point to the VulnScan platform, allowing registered users to authenticate securely using their email address and password. Upon successful authentication, users are redirected to the appropriate dashboard based on their assigned role (Developer or Administrator), ensuring role-based access control and secure access to system functionalities.

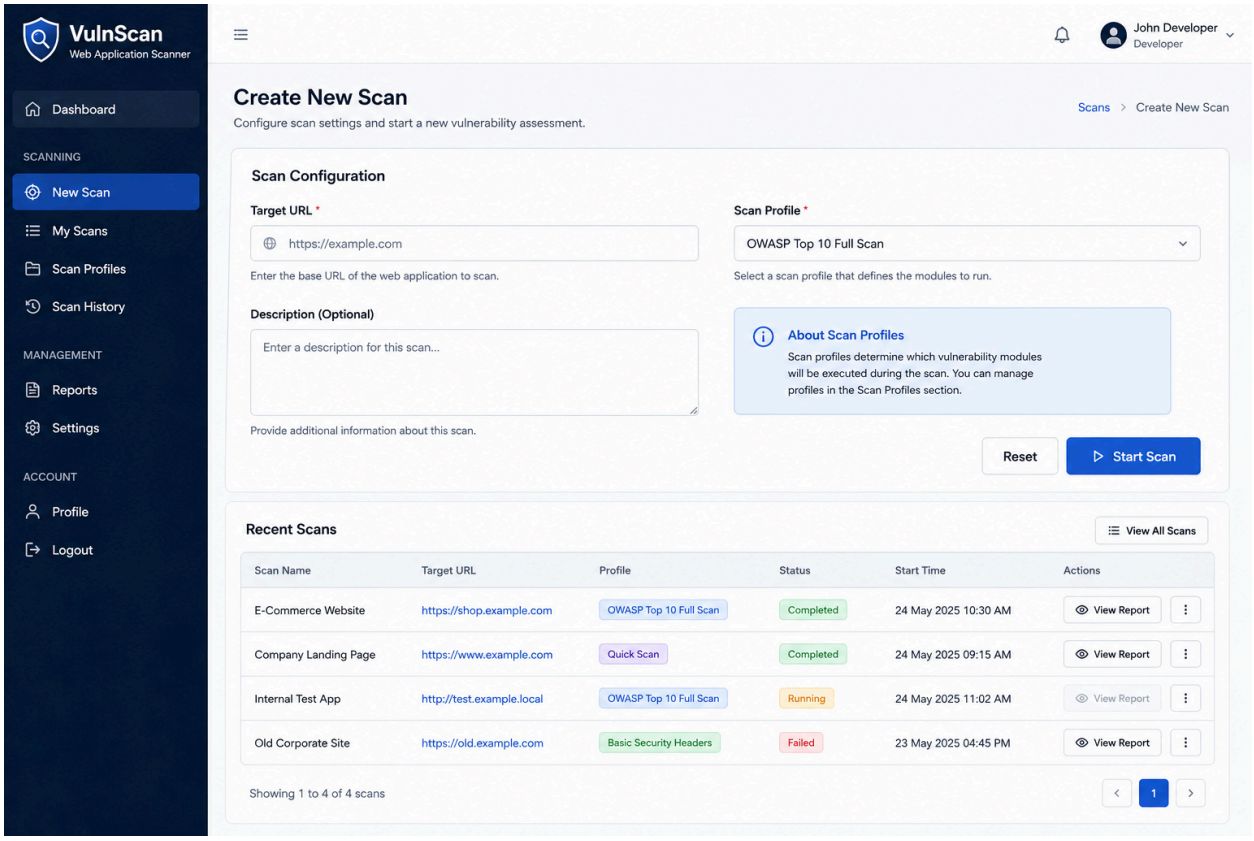



**Figure 4.4:** Developer Dashboard

**Purpose:**

This interface illustrates the primary workspace used by Developers to configure vulnerability scans, monitor scan history, and access security reports. It demonstrates how users create new scans by selecting a target URL and scan profile while providing quick access to previous scan results and scan management features.

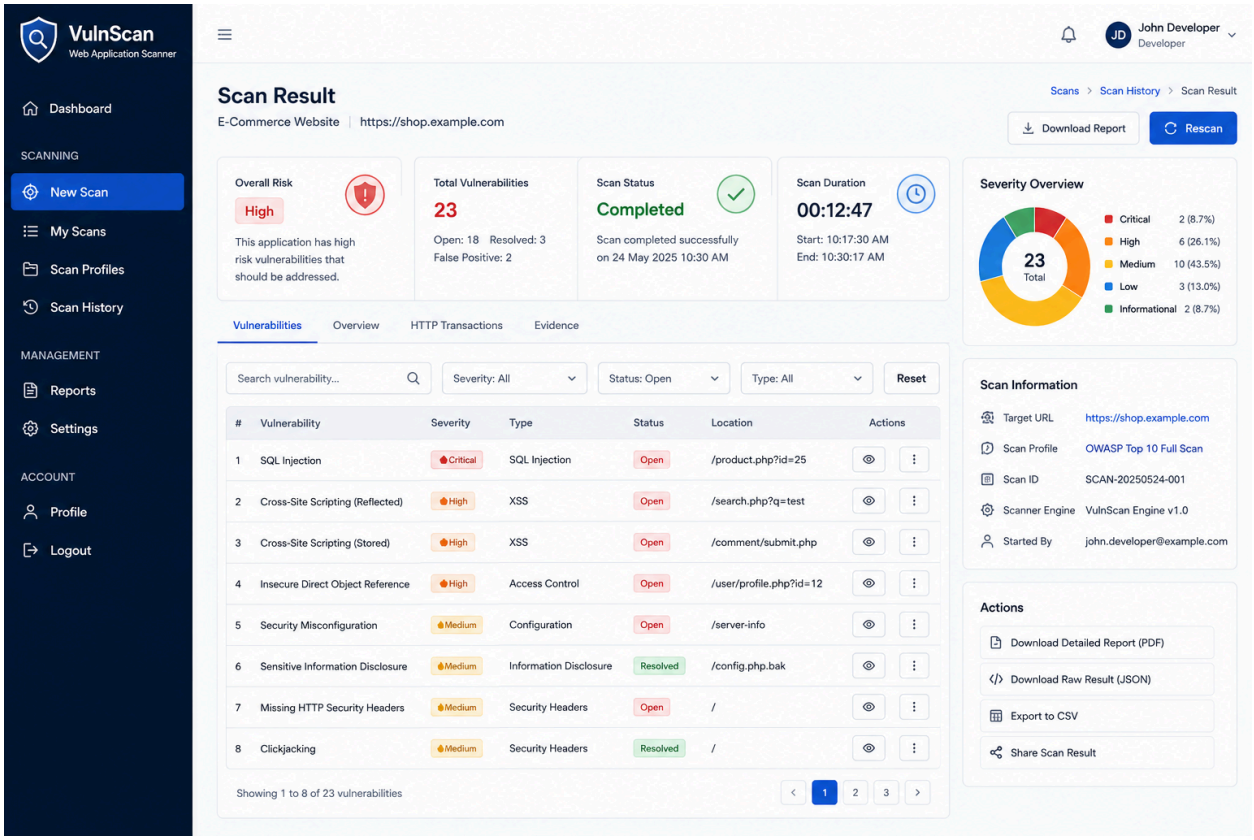



**Figure 4.5:** Scan Result Page

**Purpose:**
This interface presents the results of a completed vulnerability assessment, allowing developers to review detected vulnerabilities, their severity levels, affected locations, supporting evidence, and recommended remediation actions. It also provides summary statistics, scan information, filtering capabilities, and report export options to assist users in analysing and documenting the security assessment results.

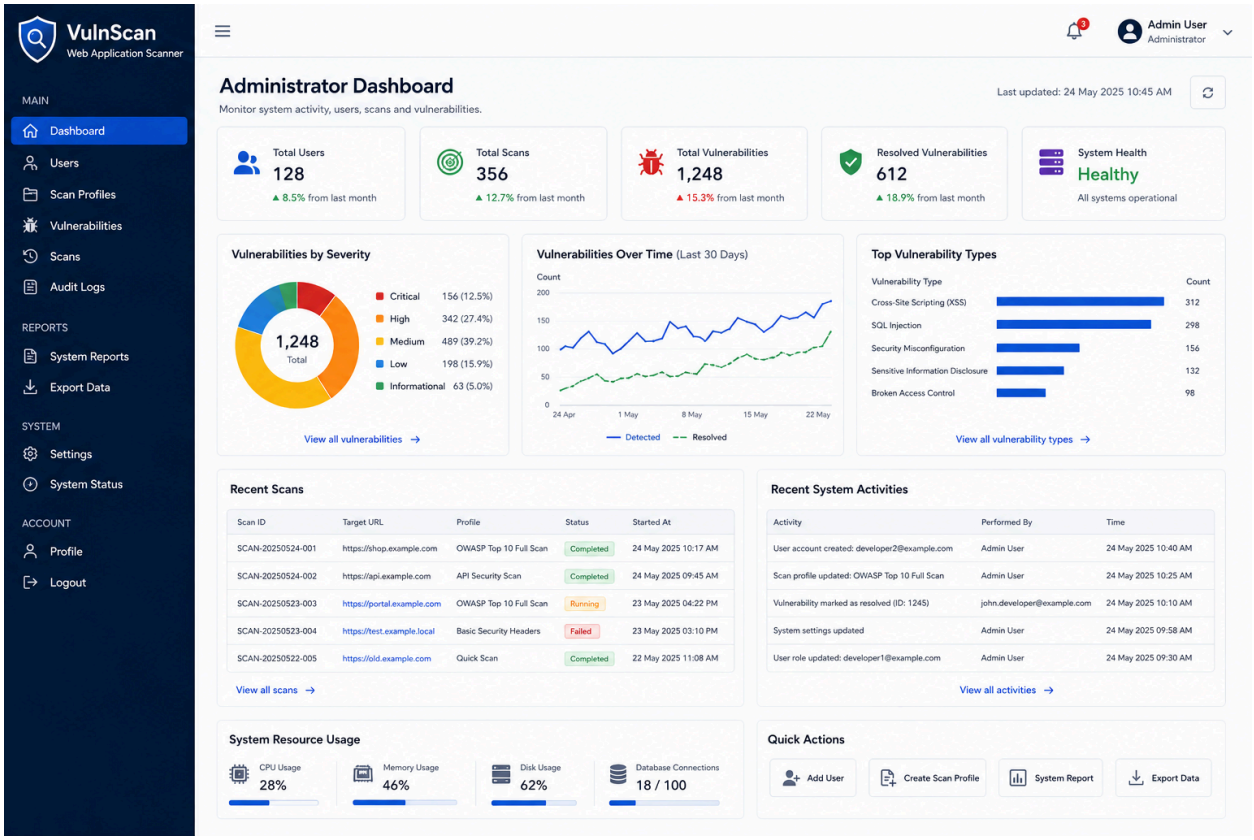



**Figure 4.6:** Administrators Dashboard

**Purpose:**
This interface provides administrators with a centralized dashboard for monitoring and managing the overall operation of VulnScan. It displays key system metrics, recent scanning activities, vulnerability statistics, system health, and provides access to administrative functions such as user management, scan profile management, vulnerability management, audit log monitoring, system reports, and data export.

## 4.4 Detailed Design

### 4.4.1 User Authentication Process

The user authentication process ensures that only authorised users can access the VulnScan platform. When a user submits their login credentials, the request is forwarded to the Flask REST API, where the credentials are validated against the information stored in the User entity. Passwords are verified using secure hashing techniques to protect user credentials during authentication.

Upon successful authentication, the system generates a JSON Web Token (JWT) that is returned to the client. The JWT is included in subsequent requests to verify the user's identity and assigned role. Based on the authenticated role, the system redirects the user to either the Developer Dashboard or the Administrator Dashboard, ensuring that users only access functions relevant to their responsibilities. Failed authentication attempts are recorded in the SecurityAuditLog to support system monitoring and security auditing.

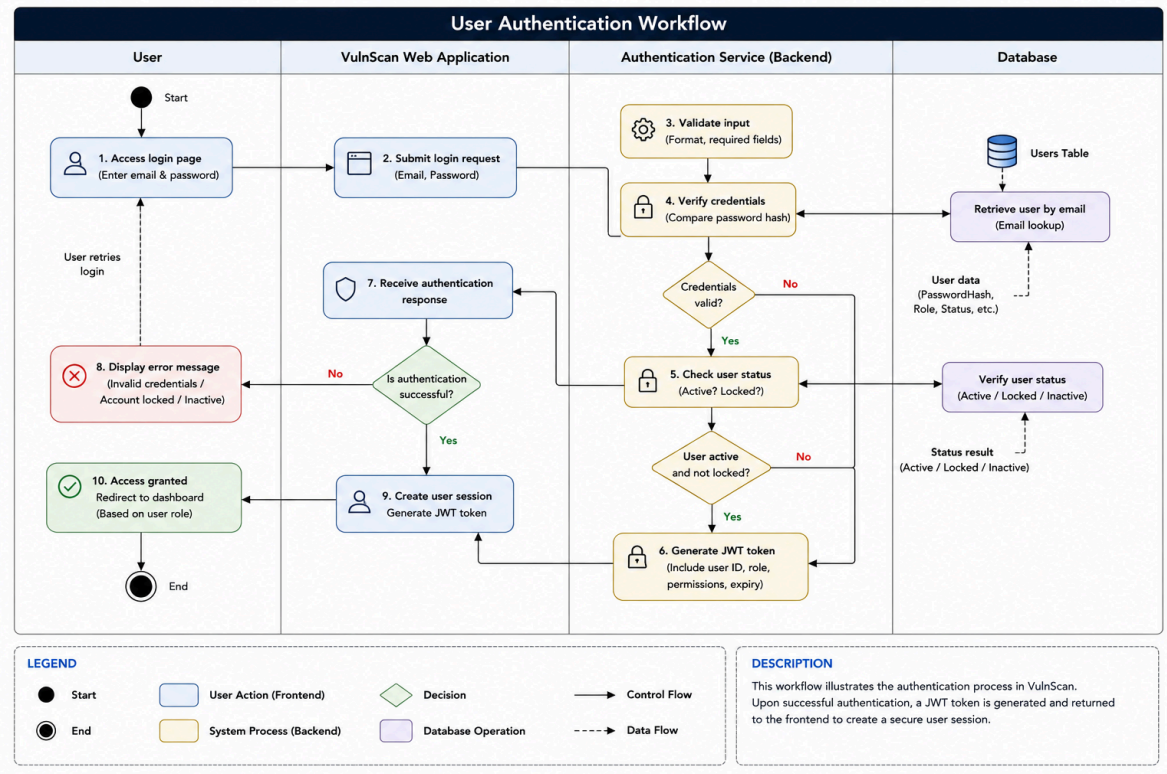



**Figure 4.7:** User Authentication Activity Diagram

**Purpose:**

This workflow illustrates the authentication process performed when a user logs into VulnScan. It shows how login credentials are validated, how the system verifies the user's account status against the database, generates a JSON Web Token (JWT) upon successful authentication, and redirects the user to the appropriate dashboard based on their assigned role, while failed authentication attempts result in an error response.

### 4.4.2 Vulnerability Scanning Process

The vulnerability scanning process begins when a developer creates a new scan by providing a target website URL and selecting an appropriate scan profile. The selected scan profile determines which vulnerability detection modules will be executed during the assessment.

The Scanner Engine first activates the Web Crawler to discover accessible pages within the target website. The discovered pages are recorded in the DiscoveredPage entity before being analysed by the enabled vulnerability detection modules. During the current implementation, the Scanner Engine supports SQL Injection (SQLi), Cross-Site Scripting (XSS), and HTTP Security Header analysis. Any identified vulnerabilities are stored as findings together with their severity, affected location, supporting evidence, and remediation recommendations.

After the scanning process is completed, the scan status is updated and the results are presented to the user through the Scan Result interface. The modular design of the Scanner Engine allows additional vulnerability detection modules to be incorporated in future versions without modifying the overall system architecture.

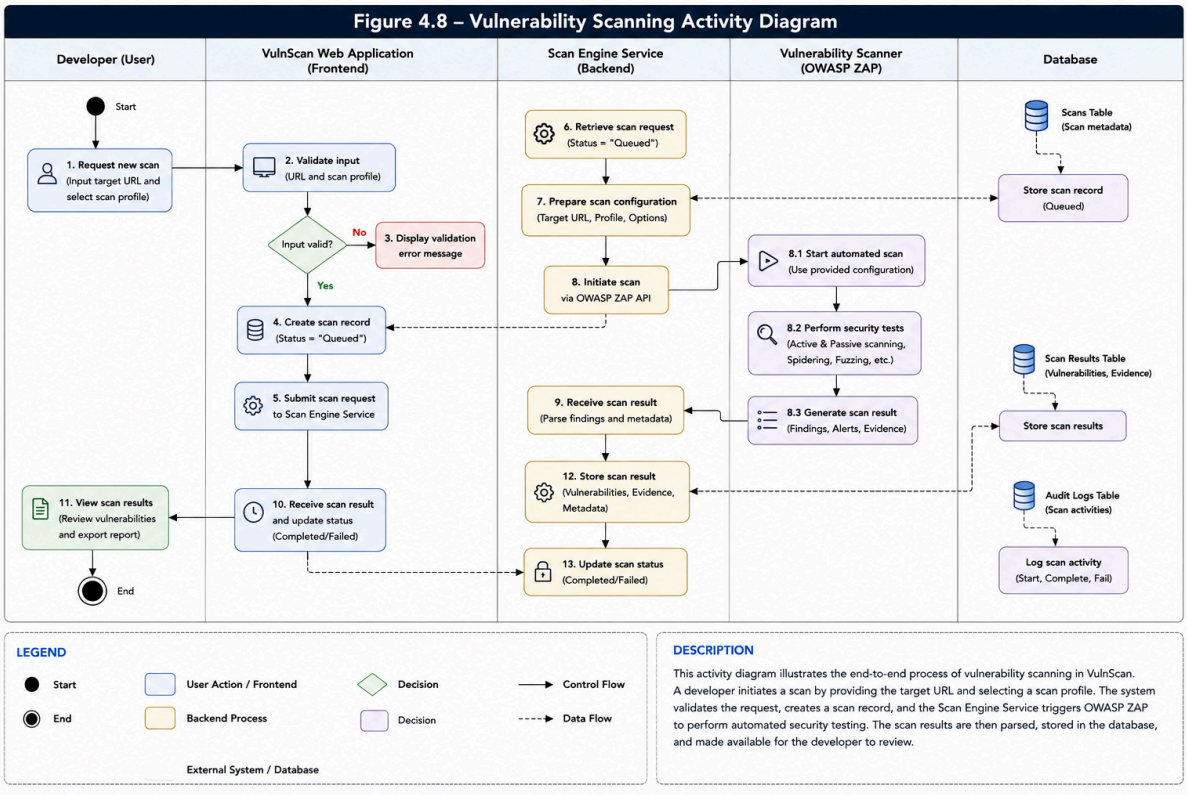



**Figure 4.8:** Vulnerability Scanning Activity Diagram

**Purpose:**

This activity diagram illustrates the end-to-end vulnerability scanning process performed by VulnScan. It shows how a developer initiates a scan, how the system validates the request, executes the configured vulnerability detection modules through the scanner engine, stores the scan results in the database, updates the scan status, and finally presents the findings to the user for analysis and reporting.

### 4.4.3 Security Audit Logging Process

To improve system accountability and security monitoring, VulnScan records important user and system activities using the SecurityAuditLog entity. Whenever significant events occur, such as user authentication, scan creation, vulnerability management, or administrative operations, the Business Logic Layer automatically generates audit records containing the event description, event time, responsible user, and related system information.

The audit logging mechanism enables administrators to monitor system activities, investigate security incidents, and maintain a historical record of important operations. Centralising these records also supports future maintenance and forensic analysis by providing a complete history of significant events performed within the application.

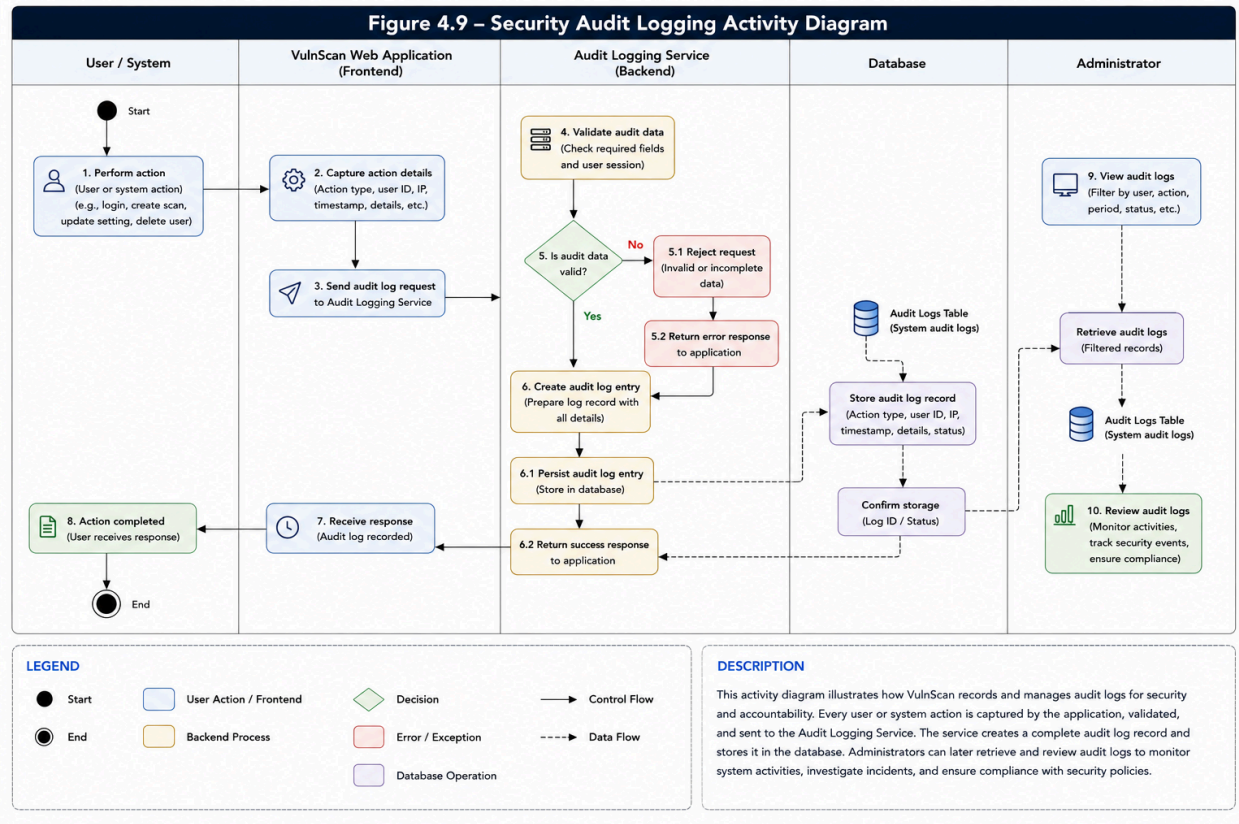



**Figure 4.9:** Security Audit Logging Activity Diagram

**Purpose:**

This activity diagram illustrates how VulnScan records and manages security audit logs throughout the system. It shows how user and system actions are captured, validated, stored in the SecurityAuditLog entity, and made available for administrators to review, supporting accountability, security monitoring, and future forensic analysis.

## 4.5 Chapter Summary and Evaluation

*This chapter presented the overall system design of VulnScan, which serves as the blueprint for the implementation of the proposed web application vulnerability assessment platform. The design was developed based on the functional and non-functional requirements identified in Chapter 3, ensuring that the proposed system architecture satisfies the project's objectives while supporting maintainability, scalability, and security.*

*The chapter first described the overall system architecture, illustrating how the Presentation Layer, API Layer, Business Logic Layer, Scanner Engine, and Data Layer interact to provide the core functionalities of VulnScan. It then presented the database design, including the Entity Relationship Diagram (ERD) and the purpose of each database entity. The user interface design was subsequently discussed to demonstrate how Developers and Administrators interact with the system through a consistent and user-friendly interface. Finally, the detailed design explained the major system processes, including user authentication, vulnerability scanning, and security audit logging.*

*Overall, the proposed system design provides a structured and modular foundation for implementing VulnScan. The design decisions presented in this chapter support future system enhancements while ensuring that the application remains maintainable and extensible. The next chapter describes the implementation of the proposed system based on the designs presented in this chapter.*

Chapter 5

# Implementation and Testing

# 5.0 Implementation and Testing

A short introduction that describes what will be included in this chapter.

IMPORTANT NOTE TO STUDENTS: Include details about:

- **Implementation**

  A detailed description of how you actually carried out the implementation (e.g., coding, etc.) of your system/prototype.

  - Include code snippets and descriptions to show how the requirements of the application/prototype have been met –
    - For Smart Campus projects, code snippets of the MQTT protocol for both client side and server side has to be included with explanation.
  - Include descriptions of important settings - Setting for server, network protocol, IP, database, security
  - Note: this should **not** be a chronological account of the work you carried out.

- **Testing**

  All test cases that have been carried out must be provided - To tabulate the test cases in tables

Note: Students may also opt to split Implementation and Testing into 2 separate chapters.

## 5.1 Sub-section 1 Heading

Sub-sections should be used to divide the chapter into logical parts.

### Sub-subsection Heading

Sub-section numbering should be limited to a maximum of 3 levels (e.g. 5.3.1) in order to avoid confusion.

## 5.2 Sub-section 2 Heading

### Sub-section 1 Heading

Sub-sections should be used to divide the chapter into logical parts.

#### Sub-subsection Heading

Sub-section numbering should be limited to a maximum of 3 levels (e.g. 5.3.1) in order to avoid confusion.

## 5.3 Sub-section 1 Heading

Sub-sections should be used to divide the chapter into logical parts.

#### Sub-subsection Heading

Sub-section numbering should be limited to a maximum of 3 levels (e.g. 5.3.1) in order to avoid confusion.

## 5.4 Chapter Summary and Evaluation

*At the end of each chapter, evaluate the contents stated or discussed in the relevant sub-sections.*

Chapter 6 *(if applicable)*

# System Deployment

# 6.0 System Deployment

This chapter would be applicable for students who have embarked on a real-life industrial project. Students in this case would need to describe how the deployment has been carried out. Some of implementation tasks which need to be described include: training, file conversion or creation, and changeovers.

## System Backup and Risk Management

Describe the procedures to backup the existing system for changeover purpose. Discuss the potential risk(s) for the changeovers and the solution.

## On-site Setup

Describe the preparations to be done prior to the setup of the new system on client's site. Discuss the procedures to setup the new system, schedule, etc.

## Training Procedure

Describe the procedures on the training procedures, contents, schedule, etc.

## Follow-up

Describe the plan or procedures to follow up with the client (company) to verify the system reliability.

## Chapter Summary and Evaluation

*At the end of each chapter, evaluate the contents stated or discussed in the relevant sub-sections. For example,*

- Problems faced. Describe the various problems faced by students in the course of doing the project.
- Solutions. What have been done to solve the problems?
- What tools and techniques have been used and reasons for using them.

Chapter 7

# Discussions and Conclusion

# 7.0 Discussions and Conclusion

Each student is required to make an *evaluation of the project* he/she has embarked on. The project evaluation may include the following sections.

IMPORTANT NOTE TO STUDENTS: In this chapter, for problems related to code, hardware, internet connection, etc (where applicable):

- List the technical problems faced and state how they were resolved
- List the unsolved technical problems for future enhancement
- List the achieved objectives/modules
- List the incomplete parts for future enhancement - this is regardless the parts listed in the pre-determined scope. Suggestions for future improvement

## Summary

Summarize the project including the problem and proposed solutions, justification of the choice of tools, techniques and methodologies used in this project.

## Achievements

Students are required to evaluate the project's achievement against project objectives, completion of the project, students' view of the strengths and weaknesses of the work done.

## Contributions

Discuss the creativity, innovativeness, contribution of the proposed system. Explain why the proposed system is necessary. Describe the marketability of the system.

## Limitations and Future Improvements

Identify the limitations of the research or project. Provide suggestions for improvement or further development of the system or research in the future.

## Issues and Solutions

Students are required to describe the various problems faced by students during the project development and explain what has been done to solve the problems. The valuable experiences gained or lessons learnt through the project as a whole. The issues may include technical issues, project management issues, team dynamics problems, and other difficulties encountered and lessons learnt. How the issues are solved or can be solved to ensure the project can be completed on time or to be improved in the future.

# References

https://owasp.org/www-project-top-ten/

https://csrc.nist.gov/pubs/sp/800/53/r5/upd1/final

https://portswigger.net/burp/documentation

https://www.zaproxy.org/

https://www.first.org/cvss/

https://csrc.nist.gov/projects/role-based-access-control

https://portswigger.net/burp/communitydownload

https://www.acunetix.com/

https://agilemanifesto.org/

publication, etc.), should also be given. Web references can be listed separately (e.g., after the reference list) under a different heading if desired, or can be included in the reference list. The list of references should contain all the items that have been explicitly referred to in the report. Students may have referred to some manuals or conducted research on certain knowledge areas, for example Oracle, RFID scanners etc.

If the students choose to write a bibliography, then they should list down all materials that are useful either directly or indirectly to the project concerned and which the students have read. An example of material useful could be an article or an Oracle user manual etc.

References should be cited in the main text using Author-Year system. The full references should be given as below (essentially APA Referencing format), in the alphabetical order in 10 pt. Times New Roman, with a 6pt spacing between each. References should be arranged first alphabetically and then further sorted chronologically if necessary. More than one reference from the same author(s) in the same year must be identified by the letters "a", "b", "c", etc., placed after the year of publication.

Please ensure that every reference cited in the text is also present in the reference list (and vice versa). Any references cited in the abstract must be given in full.

*Web references* used as a minimum, the full URL should be given and the date when the reference was last accessed. Any further information, if known (DOI, author names, dates, reference to a source

# Appendices

In order to enhance better understanding of the project, students should as far as possible include all directly relevant materials, figures or diagrams in the **main body** rather than in the Appendix. The appendix is reserved only for items which may not directly be relevant or essential to enhance a reader's understanding of the project, or which may interrupt the smooth reading of the project document (for example being too voluminous).

Appendices should only include supportive materials **directly referred** to in the writing and should be kept to a **minimum**, e.g. selected pages of an annual report, not the entire document. Examples of items included in Appendices are:

- Company's report and documentation, such as sample invoice, purchase order form, etc.
- Project meeting documentation e.g. minutes of meetings, tracking documents, memos etc.
- Questionnaires and results, interview questions and results, pilot test and results, observation sheet and results, experiment test plan and results, etc.
- Analysis/design diagrams (only those not incorporated in the main body of the report).

If there is more than one appendix, they should be identified as A, B, etc (e.g. Appendix A). Formulae and equations in appendices should be given separate numbering: Eq. (A.1), Eq. (A.2), etc.; in a subsequent appendix, Eq. (B.1) and so on. Similarly for tables and figures: Table A.1; Fig. A.1, etc.

IMPORTANT NOTE TO STUDENTS

**APPENDIX *n* User Guide**

- List the username and password of multiple roles (if applicable)
- Provide clear screen shots of each page, explain the functions of each button

As a rough guide, the user guide should include the following sections:

**System Document**

In this section, students should provide the following pieces of information:

- **System (hardware and software requirements)**. Students should describe the minimum hardware and software requirements to install the software application which has been developed by the students, for example, DBMS, OS, program development tools etc.

- **Installation**. Under this section, students should create an 'installation' CD and provide a brief step-by-step guide on how a new user can install the software on a computer system. Students should indicate any special setup information, such as the specific location of placing the database files, the SQL statements to add tables, etc. Software source code should also be included in this CD. Students are not required to print out the software code.

**Operation Document**

Under this section, students are required to provide a brief step-by-step guide on how to use the installed software. The guides should teach the user how to run the software and use its major functions and features. For example, steps show guide the users on how to run the system, e.g. to use an executable file or to use the IDE. The login information such as username and password for each of the different users (or roles) must be provided. Some screen interfaces would be useful.

## APPENDIX *n+1* Developer Guide*

*To be included for ***Smart Campus Projects*** and ***Real-Life Projects***. This section should include the following:

- List the necessary software, installer, API, library that must be installed

- List the authentication details, such as username and password for all security

## Other Appendices:

- Supporting documents
- Non-disclosure agreement (if applicable)
- Other detailed documentation, such as important references, interview results, survey results, etc.

This page is intentionally left blank to indicate the back cover. Ensure that the back cover is black in color.